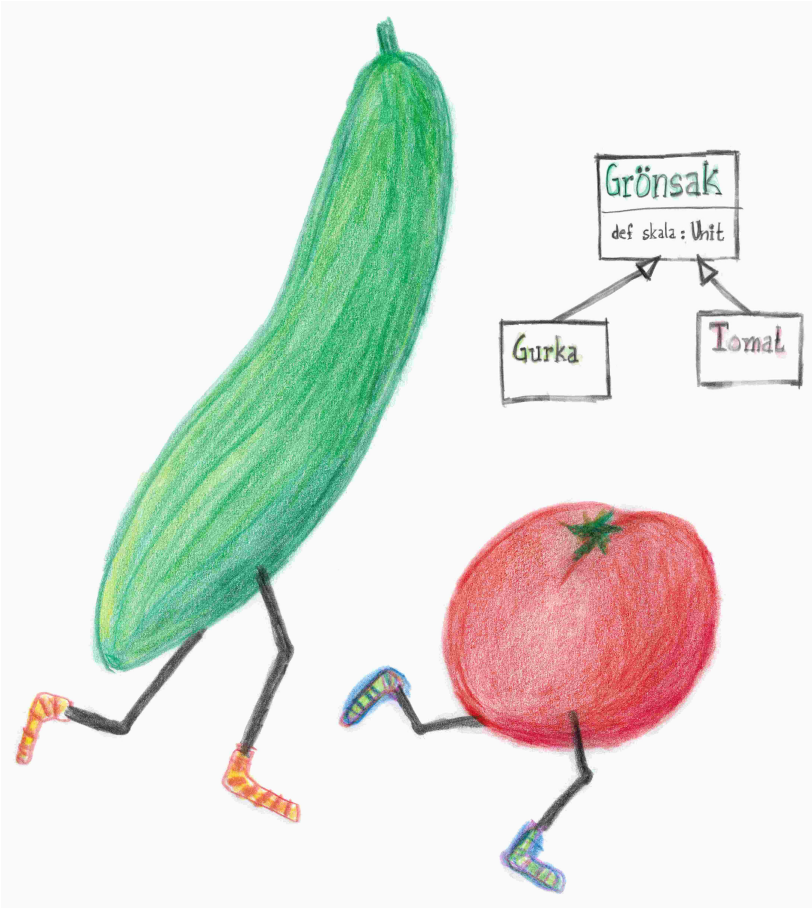
Introduction to programming with Scala

Labs and projects



H-ICT-SE-2-25 2026
Object Oriented Software Development
The Hague University of Applied Science

Compilation date: March 29, 2026
https://lunduniversity.github.io/pgk

Editor: Björn Regnell

Contributors in alphabetical order: Anders Buhl, André Philipsson Eriksson, Anna Axelsson, Anna Palmqvist Sjövall, Annie Predel, Anton Andersson, Benjamin Lindberg, Björn Regnell, Casper Schreiter, Cecilia Lindskog, Dag Hemberg, Elias Åradsson, Elliot Bräck, Elsa Cervetti Ogestad, Emelie Engström, Emil Wihlander, Erik Bjäreholt, Erik Grampp, Evelyn Beck, Felix Ohrgren, Fredrik Danebjer, Fritjof Bengtsson, Gustav Cedersjö, Henrik Olsson, Hjalmar Rutberg, Hussein Taher, Jakob Hök, Jakob Sinclair, Johan Ravnborg, Johannes Nydahl, Jonas Danebjer, Jos Rosenqvist, Lovisa Löfgren, Maj Stenmark, Maria Kulesh, Melker Widén, Måns Magnusson, Nicholas Boyd Isacsson, Niklas Sandén, Oliver Levay, Oliver Persson, Oscar Sigurdsson, Oskar Berg, Oskar Widmark, Patrik Persson, Per Holm, Philip Sadrian, Sandra Nilsson, Sebastian Hegardt, Simon Persson, Stefan Jonsson, Theodor Lundqvist, Tim Borglund, Tom Postema, Valthor Halldorsson, Viktor Claesson, Wilhelm Wanecek, William Karlsson.

Home: https://lunduniversity.github.io/pgk

Repo: https://github.com/lunduniversity/introprog

This compendium is on-going work.
Contributions are welcome!
Contact: bjorn.regnell@cs.lth.se

Versions:
Scala 3.7.2
JDK 21
introprog-scalalib 1.4.0

You can use this work if you respect this **LICENSE**: CC BY-SA 4.0
http://creativecommons.org/licenses/by-sa/4.0/
Do *not* distribute solutions to lab assignments and projects.

Copyright © 2015-2026.
Björn Regnell, Dept. of Computer Science, LTH, Lund University.

Contents

I Labs of the first study period **1**

1 Lab: `kojo` **3**
1.1 Mandatory tasks . . . 3
1.2 Checkpoint questions . . . 7
1.3 Optional extra exercises . . . 7

3 Lab: `irritext` **13**
3.1 Requirements . . . 13
3.2 Tips to get started . . . 14
3.3 Inspiration . . . 14

4 Lab: `blockmole` **17**
4.1 Background . . . 17
4.2 Mandatory tasks . . . 17
4.3 Control questions . . . 22
4.4 Optional extra tasks . . . 23

5 Lab: `blockbattle0` **27**

6 Lab: `blockbattle1` **29**
6.1 Background . . . 30
6.2 Mandatory requirements . . . 30
6.3 Optional requirements – choose at least one . . . 31
6.4 Preparations for presentation . . . 31
6.5 Tips and suggestions . . . 31

7 Lab: `shuffle` **35**
7.1 Background . . . 35
7.2 Given code . . . 36
7.3 Mandatory tasks . . . 37
7.4 Optional extra tasks . . . 41
7.5 Pictures with examples of different poker hands . . . 41

II Labs and projects of the second study period **43**

8 Lab: `life` **45**
8.1 Background . . . 46
8.2 Mandatory requirements . . . 47
8.3 Optional requirements – choose at least one . . . 47
8.4 Tips and suggestions . . . 48

9 Lab: words **51**
9.1 Background 51
9.2 Mandatory tasks 52
9.3 Control questions 57
9.4 Optional tasks 57

10 Team lab: snake0 **59**
10.1 Background 60
10.2 Mandatory functional requirements 60
10.3 Mandatory design requirements 62
10.4 Optional requirements – each person must choose at least one 65
10.5 Tips and suggestions 66

11 Lab: snake1 **71**
11.1 Group Lab Presentation 71

12 Project **73**
12.1 Project Assignment: bank 74
12.2 Project Assignment: music 82
12.3 Project Assignment: photo 90

Part I

Labs of the first study period

Chapter 1

Lab: `kojo`

Goals

- ☐ Be able to apply and combine the principles of sequence, alternative, repetition, and abstraction in creating own programs of at least 20 lines of code.
- ☐ Be able to explain what a program does in terms of sequence, alternative, repetition, and abstraction.
- ☐ Be able to format own programs so that they become easy to read and understand.
- ☐ Be able to explain what a variable is and be able to declare immutable and mutable variables, as well as make assignments.
- ☐ Be able to carry out repeated rounds in the cycle edit-execute-debug/improve to gradually build up increasingly developed programs.

Preparations

- ☐ Review this week's lecture materials.
- ☐ Do exercise `Excercise01` in section 1.
- ☐ Read about Kojo in appendix A.
- ☐ Read through the entire lab below. Consider possible solutions for the tasks that are marked with a pencil in the margin.
- ☐ Fetch the given code via the course GitHub repository.

1.1 Mandatory tasks

If there is a pencil in the margin, you should make notes in preparation for the presentation.

Task 1. *Install and start Kojo.* Open the terminal and enter the following commands to download the file `kojo.scala` from the internet, save it in the directory `~/w01-kojo`, and start Kojo.

```
> mkdir ~/w01-kojo
> cd ~/w01-kojo
> curl -sLO https://fileadmin.cs.lth.se/kojo.scala
> scala repl ~/w01-kojo/kojo.scala
```

You should now be able to write `activateCanvas()` in the terminal, press ENTER, and see a window appear on the screen.

```
Welcome to Scala 3.7.3 (21.0.8, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> activateCanvas()

scala>
```

Task 2. *Sequence and repetition*. Draw a square using `repeat(n){ ??? }` where you replace `n` with the number of repetitions and `???` with the statements to repeat.

Task 3. *Variable and repetition*.

a) The function `System.currentTimeMillis` is part of Java's standard library and returns an integer of type `Long` with the current number of milliseconds since midnight on January 1, 1970. With the Kojo procedure `setAnimationDelay(0)` there is no delay when the turtle draws and the drawing happens as fast as possible. Enter the following program in the REPL and explain what happens. *Note:* hold down `ESC` while pressing `ENTER` to create a line break without running the program.

```
clear()
animationDelay(0)
val n = 800 * 4
val t1 = System.currentTimeMillis
repeat(n):
  repeat(4):
    forward
    right
val t2 = System.currentTimeMillis
println(s"$n square rounds took ${t2 - t1} milliseconds")
```

b) Note approximately how many square rounds per second the turtle can draw at its fastest. Run several times because the virtual machine needs to "warm up" for the machine code to be optimized. Some runs may be slower if the garbage collector needs to spend time freeing memory. ✎

c) What are the names of the variables in the code above? What values do the variables have? ✎

d) Draw a square again, but now using a **`while`** statement and a loop variable.

```
slow(100)
var i = 0
while ??? do
  forward
  right
  i = ???
```

e) What is the difference between variables declared with **`val`** and variables declared with **`var`**? ✎

f) Draw a square again, but now using a **`for`** statement. Print the value of the local variable `i` in each loop iteration.

```
def square =
  for ??? do
    ???

for ??? do
  ???
```

Figure 1.1: A column of squares.

```
for i <- 1 to ??? do
  ???
```

g) Is it possible to assign a new value to the variable `i` inside the loop?

h) Is it possible to refer to the name `i` outside the loop?

i) Draw a square again, but now using `foreach`. Print the loop variable's value in each iteration.

```
(1 to ???).foreach: i =>
  ???
```

Task 4. *Abstraction*.

a) Use a repetition to abstract the sequence below so that the program becomes shorter:

```
forward; right; jump; forward; left; jump; forward; right;
jump; forward; left; jump; forward; right; jump; forward;
left; jump; forward; right; jump; forward; left; jump;
forward; right; jump; forward; left; jump
```

b) Define your own procedure named `square` using the keyword **`def`** which, when called, draws a square using a **`for`**-loop.

```
def square =
  for ??? do
    ???
```

c) Call your abstraction after it has been declared and after you have executed: `setAnimationDelay(100)`

d) Call your abstraction inside a **`for`**-loop so that the turtle draws a column consisting of 10 squares as shown in the image below.

e) Examine how calling the procedure `square` affects the execution sequence of your statements by making appropriate printouts so you can see when different parts of the

code are executed. At which points in the program does a ”jump” in the sequence occur instead of the following statement being executed? Use an appropriate argument to `setAnimationDelay` so that you have time to study the execution.

f) Draw the same picture with 10 stacked squares (see figure 1.1 on page 5), but now *without* using the abstraction `square` – instead use a nested repetition (i.e., a repetition inside a repetition). Which of the two ways (with and without the abstraction `square`) is easiest to read?

g) Generalize your abstraction `square` by giving it a parameter `side: Double` that specifies the square's size. Draw several squares as in figure 1.2 on page 6.

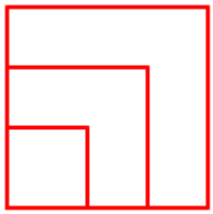


Figure 1.2: Squares of different sizes.

Task 5. *Alternative.*

a) Run the program below. Explain what happens.

```
setAnimationDelay(5000)

def move(key: Int): Unit =
  println("key: " + key)
  if key == 87 then forward(10)
  else if key == 83 then forward(-10)

move(87); move('W'); move('W')
move(83); move('S'); move('S'); move('S')
```

b) Run the program below. Note the `activateCanvas()` call so you don't need to click inside the drawing window before you can control the turtle. The call `onKeyPress(move)` makes `move` be called when a key is pressed. Add code in `move` so that the key A causes a 5-degree left rotation while the key D causes a 5-degree right rotation. With `onKeyPress` you define which procedure should run upon keypress.

```
setAnimationDelay(0)
activateCanvas()

def move(key: Int): Unit =
  println("key: " + key)
  if key == 'W' then forward(10)
  else if key == 'S' then forward(-10)

onKeyPress(move)
```

1.2 Checkpoint questions

✓ 👁 Review the theory for this week and be ready to answer these questions when it is your turn to present what you have done during the lab:

1. What does sequential execution of statements mean?
2. What is the difference between a statement and an expression?
3. What is the difference between a procedure and a function?
4. Does the order of arguments matter when calling a function with multiple parameters?
5. What is a variable? Give examples of declaration, initialization, and assignment of variables, as well as usage of variables in expressions.
6. What is a logical expression? Give examples of usage of logical expressions.
7. What is abstraction? Give examples of use of abstraction.
8. What is the benefit of abstraction?
9. How is a variable declared and initialized whose value is mutable?
10. How is a variable declared and initialized whose value is immutable?
11. Is it a runtime error or a compile-time error to assign a new value to an immutable variable?
12. Indicate which of `for` and `while` is most appropriate in these cases:
    A. Sum the first hundred integers.
    B. Count the number of characters in a string until the first space character.
    C. Draw 100 random numbers between 1 and 6 and sum those numbers less than 3.
    D. Sum the integers starting from 1 upwards until the sum is at least 100.

1.3 Optional extra exercises

Do the following tasks in any order if you wish and depending on your exercise needs.

Task 1. *Abstraction and generalization*.

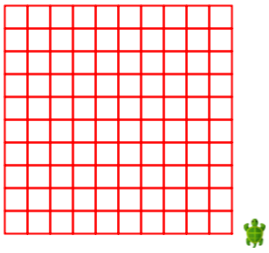
a) Create an abstraction `def stapel = ???` that uses your abstraction `square`.

b) Now you will *generalize* your procedure so that it does not just draw exactly 10 squares in a column. Give the procedure `stapel` a parameter `n` which controls how many squares are drawn.

```
def square = ???
def stapel(n: Int) = ???

animationDelay(100)
stapel(42)
```

c) Draw the image below using the abstraction `stapel`. There are a total of 100 squares and each square has side length 25. *Tip:* With a negative argument to the procedure `hop` you can make the turtle jump backwards without drawing, e.g. `hop(-10*25)`



d) Generalize your abstractions `square` and `stapel` so that you can control the size of the squares that are drawn.

e) Create an abstraction `grid` with suitable parameters so that one can draw grids with varying square sizes and numbers of squares in both horizontal and vertical directions.

f) Generalize your abstractions `square` and `stapel` so that one can influence the fill color and pen color for the squares that are drawn.
The colors in Kojo are of type `java.awt.Color`. The type is accessible under the name `Color` because the name is made directly available with **`export`** `java.awt.Color` in the file `kojo.scala` (more about the keywords **`export`** and **`import`** in reading week 4).

Task 2. *Toggling with Boolean values.*

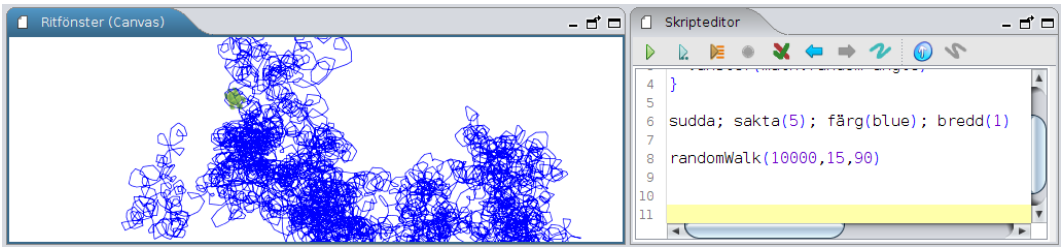
a) Build upon the program in task 5 and add the code below at the beginning of the program. Also add code so that when pressing the G key, the grid is toggled on and off alternately. Note that exactly *one* procedure is called at `onKeyPress`.

```
var isGridOn = false

def toggleGrid =
  if isGridOn then
    gridOff
    isGridOn = false
  else
    gridOn
    isGridOn = true
```

b) Make it so that when pressing the X key, the coordinate axes are toggled on and off alternately. Use a variable `isAxesOn` and define an abstraction `toggleAxes` which calls `axesOn` and `axesOff` similarly to the previous task.

Task 3. *Repetition.* Write a procedure `randomWalk` with this signature:
`def randomWalk(n: Int, maxStep: Int, maxAngle: Int): Unit`
which makes the turtle take `n` steps of random length between `0` and `maxStep`, and after each step turn left by a random angle between `0` and `maxAngle`. Call your procedure with different arguments and examine how the values affect the image's appearance. *Tip:* The expression `math.random() * 100` gives a number from 0 to (almost) 100. You can control how slowly the turtle draws by calling `setAnimationDelay(???)` (try to find an appropriate integer argument instead of `???`).



Task 4. *Variables, naming and formatting.*

a) Paste the strangely formatted program below *exactly* as it is with whitespaces, indentations, and line breaks. Run the program and explain what happens.

```
// A strangely formatted program with some strange names.

def cucumber(x: Double, y: Double, name: String, type: String, value:String) = {
val tomato = 15
val h = 30
setPosition(x,y)
north
write(name+": "+type)
setPosition(x+tomato*(name.size+type.size),y)
write(value); south; front(h); left
front(tomato * value.size); left
front(h); left
front(tomato* value.size); left }
erase; color(black);
val s = 130
val h = 40
var x = 42; cucumber(10, s-h*0, "x","Int", x.toString)
var y = x; cucumber(10, s-h*1, "y","Int", y.toString)
x = x + 1; cucumber(10, s-h*2, "x","Int", x.toString)
cucumber(10, s-h*3, "y","Int", y.toString); invisible
```

b) Write down the names of all variables that appear in the program.

c) Which of these variables are local?

d) Which of these variables can change after initialization?

e) Suggest three changes to the program above (for example, renaming) that make it easier to read and understand.

f) Do search-and-replace of `cucumber` to a better name.

g) Do automatic formatting of the code using an appropriate shortcut key. Note the differences. Which autoformatting changes make the program easier to read? Which manual formatting changes do you think should be done to increase readability? Give the function `cucumber` a better name. Discuss the readability with a supervisor.

Task 5. *Timing.* How fast is your computer?

a) Enter the code below in the REPL and run repeatedly by typing `main`. Does it take the same time every time? Why?

```
object timer:
  def now: Long = System.currentTimeMillis
  var saved: Long = now
  def elapsedMillis: Long = now - saved
  def elapsedSeconds: Double = elapsedMillis / 1000.0
  def reset: Unit =
    saved = now

// MAIN PROGRAM:
def main =
  timer.reset
  var i = 0L
  while i < 1e8.toLong do
    i += 1
  val t = timer.elapsedSeconds
  println("Counted to " + i + " in " + t + " seconds.")
```

b) Change the loop in task a so that it counts to 4.4 billion. How long does it take your computer to count that far?[1]

c) If you run on a Linux machine: Run the Linux command below repeatedly in a terminal window. At what MHz is your CPU clock running currently? How does the clock frequency compare with the number of iterations in the while loop in the previous task? (It may happen that your computer can vary the CPU clock frequency. Try both while you run the timing in Kojo and while your computer is "idle". What is the benefit of a processor being able to vary its clock frequency?)

```
> lscpu | grep MHz
```

d) Change the code in task a so that the **while** loop runs only 5 times.

e) Add the code below to your program and try to find out approximately how far your computer can count in one second for `Long` and `Int` variables respectively. Write `calculateFast` in the terminal to run the main program.

```
def timeLong(n: Long): Double =
  timer.reset
  var i = 0L
  while i < n do
    i += 1
  end while
  timer.elapsedSeconds

def timeInt(n: Int): Double =
  timer.reset
  var i = 0
  while i < n do
    i += 1
  end while
  timer.elapsedSeconds

def show(msg: String, sec: Double): Unit =
```

[1] It is possible to do roughly one integer addition per clock cycle per core. The first electronic computer ENIAC had a clock frequency corresponding to 5 kHz. The computer on which this exercise was created had an i7-4790K turbo clocked up to 4.4 GHz.

```
  print(msg + ": ")
  println(sec + " seconds")

def report(n: Long): Unit =
  show("Long " + n, timeLong(n))
  if n <= Int.MaxValue then
    show("Int  " + n, timeInt(n.toInt))

// MAIN PROGRAM, measurements:
def calculateFast =
  report(Int.MaxValue)
  for i <- 1 to 10 do
    report(4.26e9.toLong)
```

f) How much faster is it to count with `Int` variables compared to `Long` variables? Discuss your answer with a supervisor if you wish.

Task 6. Play with color in Kojo. Search the internet for documentation of the class `java.awt.Color` and study which integer parameters the last constructor in the list of constructors takes to create sRGB colors. If you write `selectColor()` in the REPL, a new window opens where you can pick a color. If you then press OK or the ENTER key, the chosen color will become a value in the REPL. Try this in the REPL:

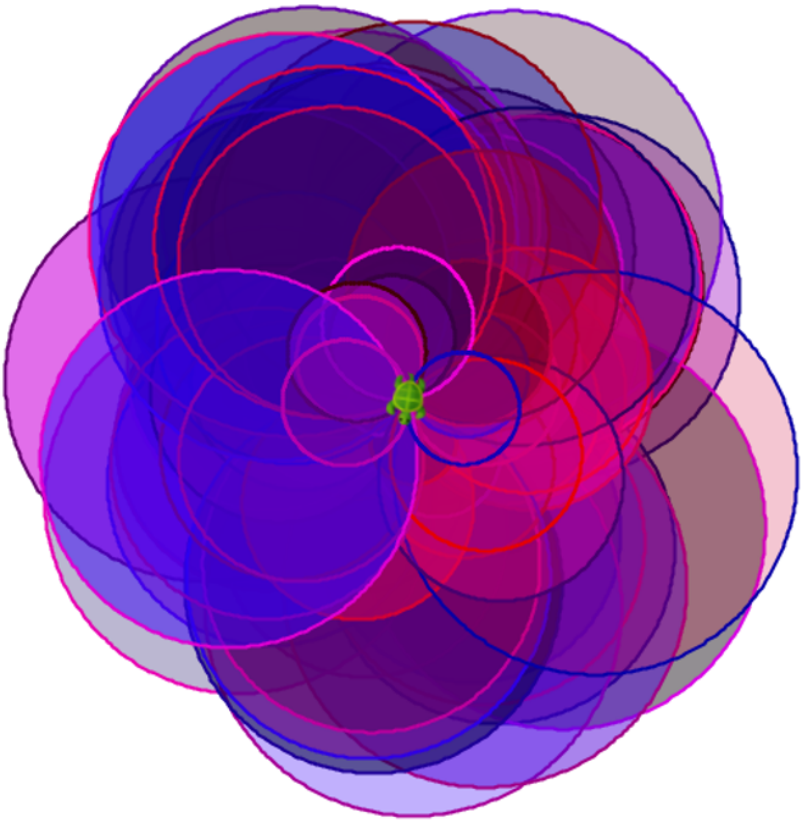
```
scala> val chosenColor = selectColor() // choose a color
val chosenColor: java.awt.Color = java.awt.Color[r=0,g=255,b=226]

scala> val c = new java.awt.Color(124,10,78,100)
val c: java.awt.Color = java.awt.Color[r=124,g=10,b=78]

scala> c.  // press TAB
asInstanceOf    getColorComponents      getRGBComponents
brighter        getColorSpace           getRed
createContext   getComponents           getTransparency
darker          getGreen                isInstanceOf
getAlpha        getRGB                  toString
getBlue         getRGBColorComponents

scala> c.getAlpha
val res3: Int = 100
```

Write a program that draws many figures with different colors, for example circles as below. If you use the alpha channel, the colors become transparent.



Task 7. Download "Missions with Kojo" from lth.se/programmera/uppdrag and do some exercises that you find interesting.

Chapter 3

Lab: `irritext`

Goals

- ☐ Be able to create a larger program with your own code based on your own ideas.
- ☐ Be able to use an editor and the terminal to iteratively edit, compile, and test your code.
- ☐ Be able to use variables in combination with conditionals and repetition at multiple levels.
- ☐ Be able to gradually improve your code to facilitate changes and increase readability.
- ☐ Be able to create and use abstractions to generalize and enable code reuse.

Preparations

- ☐ Do exercise `Excercise03` and review exercise `Excercise02` before you start the lab.
- ☐ Read appendix B and C.
- ☐ Download given code via the course's GitHub page.
- ☐ Develop a first, playable version of your text game, which you can further develop during the lab.
- ☐ Find someone who plays an early version of your game and reads your code, then gives feedback on the code's readability. Write down the feedback you receive.
- ☐ Play someone else's text game and give feedback on the code's readability.

3.1 Requirements

- You shall create a moderately annoying text game using an editor, for example VS `code` (see appendix C.1.1). The game shall run in the terminal.

- During the presentation of the lab, you shall explain which programming concepts you have practiced during the development of your text game. You shall also describe for the instructor how you have improved your code based on the feedback you received from someone who played your game and read the code.

- Your text game shall be *moderately* annoying if the player has read the code, while the game may well be unreasonably annoying for the one who *has not* read the code. It should be possible to complete the game (you decide what that means) and thereby end the program within a reasonable time having knowledge of the code.

- Try to make your code *easy to read and understand*, even if the game itself at times may be more or less incomprehensible, quirky, or troublesome for the player who does not have access to the code... Note that your code does not need to be ”perfect” from the start. Start freely and improve gradually.
- As your program becomes longer, you should reshape and split your code into many, short abstractions with well-chosen names to increase readability.
- Your code should use the important concepts that the course has covered so far, with special focus on what you specifically need to practice the most.

3.2 Tips to get started

- Create a directory that contains a Scala code file with any name.
- Write a simple `@main` method in the newly created code file that only prints the string `"Hello World!"`.
- Compile and run, fix any errors until the program works correctly.
- When the program works, begin extending the `@main` method in your code file and implement more functionality, take a look under inspiration below.
- Start simple and try to formulate what your program should do with *pseudocode* as comments before you write the code.
- Compile and run at every addition and keep each addition as small as possible, so you avoid sorting out a lot of difficult subsequent errors at compilation and any runtime errors become more understandable.
- Keep extending until the lab requirements have been met.

3.3 Inspiration

Here follows a list of different suggestions for features that you can choose from, combine, and vary in different ways. You can also include completely other features in your game. The most important thing is that you combine joy of coding with educational challenges :)

- Ask the user to log in. Give silly error messages if the user does not know the password.
- Let the user end up in an annoying endless loop of pointless questions if they make a ”wrong” choice.
- Describe a scary fantasy place where the user is located, for example a cave | a basement | a spaceship | the Chemistry Center.
- Let the user choose between silly weapons, for example a floor mop | cotton swab | foil hat | poisoned chocolate biscuit.
- Let the user choose between different paths | doors | tunnels | section cafés. Let the choice dictate which monsters are encountered. Let the user fight the monster with different weapons.

- Introduce some kind of score that is shown during the game and at the end.
- Introduce different kinds of points for health, combat power, achieved skill level, etc.
- Ask the user about more or less relevant details: name | shoe size | favorite pet. Give silly comments where these details are included as substrings.
- Play rock | paper | scissors with the user.
- Play ”guess the number” and give hints if the number is too low or too high.
- Measure how long it takes for the user to complete your game and assign points accordingly.
- Check the user’s reaction time by measuring how long it takes to press Enter after waiting a random time for the string `"NOW!"` to be printed. If the user presses Enter before the start print, the measured time is 0 and thus your program can detect that the user pressed too early. Measure reaction time repeatedly and give points based on the average.
- Let the user write different words backwards as fast as possible within a time limit.
- Ask the user to write a palindrome. Give points based on length.
- Train the user in the multiplication table against the clock.
- Let the user answer multiple-choice questions about your favorite movie.
- Make it possible to provide an extra argument with a ”cheat code” that gives the user special abilities or otherwise eases the gameplay.

Chapter 4

Lab: `blockmole`

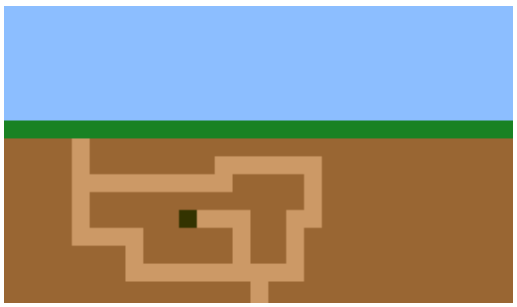
Goals

- ☐ Be able to explain how singleton objects can be used as modules.
- ☐ Be able to explain how access to members in singleton objects occurs.
- ☐ Be able to create code that reacts to and changes the state of objects.
- ☐ Be able to explain the benefit of collecting named constants in a separate module.
- ☐ Be able to explain how import affects the visibility of names.
- ☐ Be able to give an example of a situation where renaming upon import is useful.
- ☐ Be able to describe the difference between packages and singleton objects.
- ☐ Be able to create and use tuples.

Preparations

- ☐ Do exercise `Excercise04` and review exercise `Excercise03`.
- ☐ Review appendices B, C, and D.
- ☐ Obtain given code via the course's GitHub repository.

4.1 Background



Blockmole (*Talpa laterculus*) is a fantasy creature in the mole family. It is known for its distinctive square appearance. It mostly lives alone in its underground tunnels which, unlike the real mole's (*Talpa europaea*) tunnels, have completely straight walls.

4.2 Mandatory tasks

Task 1. *Create directory and code file.* Step by step, you will create a program that lets the user interact with a living blockmole. Use an editor, e.g., VS `code`, compile your program in the terminal with `scala compile . --watch` and run it in another terminal window with `scala run .`

a) Create a new file named `blockmole.scala` in a new directory in your home directory, for example `~/pgk/w04/lab/blockmole.scala`, where ~ is your home directory.

```
> mkdir -p ~/pgk/w04/lab
> code ~/pgk/w04/lab/blockmole.scala
```

b) Navigate to your new directory and check that your new file is there.

```
> cd ~/pgk/w04/lab/
> ls
blockmole.scala
```

c) Make a package declaration at the beginning of the file `blockmole.scala` so that the code you will write below is part of the package `blockmole`.

d) Then declare a single object named `Main` with a `@main def run` procedure that prints the text: `"Keep on digging!"`

e) Compile your program with `scala compile .` and look for files ending with `.class` in subdirectories of `.scala-build`. What is the name of the subdirectory with your program's machine code files? Why did this subdirectory get this name? ✎

f) Run the command `scala run . --main-class blockmole.run` to execute your program and check the output in the terminal window.

Now you have written a program that encourages a blockmole to keep digging. That program is not very useful since moles cannot read. The next step is therefore to write a graphical program.

Task 2. *Create a basic structure for the program.* In smaller programs, it works well to gather all functions in a single object, but in large programs it becomes easier to find your way in the code and understand what it does if you have several modules with different responsibilities. Your program should have the following overall structure:

```scala
package blockmole

object Color:
  // Creates different colors needed in other modules
  ???

object BlockWindow:
  // Has an introprog.PixelWindow and draws block graphics
  ???

object Mole: // Represents a blockmole that can dig
  def dig(): Unit = println("This is where the digging happens!")

object Main:
  def drawWorld(): Unit = println("Will draw the underground!")

  @main def run =
    drawWorld()
    Mole.dig()
```

Create the above program skeleton in the file `blockmole.scala` and make sure the code compiles without errors and can be run with outputs as expected. The function `???` in the skeleton is used as a placeholder so that the code can compile even though the single object's bodies are currently empty (more about this in chapter 5). Replace `???` with the actual code for Color and BlockWindow in the upcoming subtasks.
In this lab, we place all modules in the same file, but in other situations when the modules grow large and/or need to be reused by several different programs, it is good to have them in separate files so they can be compiled and tested separately.

Task 3. *Add colors in the color module.* In the single object `Color`, we will create colors using the Java class `java.awt.Color`. Since our single object name conflicts with the name of the Java color class, we rename the Java class to `JColor` in the import declaration.

a) Insert an import declaration with the renaming immediately after the package declaration. We place the import so that it is visible throughout the package because several objects need access to `JColor`. Ensure the code still compiles without errors.

b) Then create the following colors in the object `Color`:

```
object Color:
  val black  = new JColor(  0,   0,   0)
  val mole   = new JColor( 51,  51,   0)
  val soil   = new JColor(153, 102,  51)
  val tunnel = new JColor(204, 153, 102)
  val grass  = new JColor( 25, 130,  35)
```

Task 4. *Create a drawing window in the block graphics module.* Add the following three variables to the single object `BlockWindow`:

```
  val windowSize = (30, 50)  // (width, height) in number of blocks
  val blockSize  = 10        // number of pixels per block

  val window = new PixelWindow(???, ???, ???)
```

- Import `introprog.PixelWindow` locally inside `BlockWindow`. (A local import declaration is good here since only this object needs access to `PixelWindow`.)
- Make the size of `window` correspond to the block size times width respectively height in `windowSize`.
- Give the window an appropriate title, e.g., `"Digging Blockmole"`.
- When compiling, you need to make sure that `introprog` is available on the `classpath` (see exercise `Excercise04`).
- If you forgot the parameter order to the class `PixelWindow` check the documentation for `PixelWindow` [1]. Use named arguments when creating the window. Do you think the code becomes more readable with named arguments? [2]

[1] https://fileadmin.cs.lth.se/pgk/api

[2] Unfortunately you cannot use named arguments when instantiating Java classes in Scala, but PixelWindow is implemented in Scala so here it works fine.

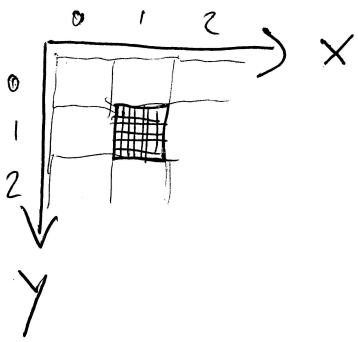


Figure 4.1: Each block consists of many pixels. The marked block has coordinate (1,1) in block coordinates, while the block's top-left pixel has coordinate (7,7) in `PixelWindow` coordinates, if e.g. there are seven-by-seven pixels per block. What is the block coordinate of the block to the right of the marked block in the picture? What are its `PixelWindow` coordinates for the top-left and bottom-right pixels?

To test the window, add a simple test drawing by using `BlockWindow.window` in procedure `drawWorld`, e.g.:

```
def drawWorld(): Unit =
  BlockWindow.window.line(100, 10, 200, 20)
```

Compile and run and ensure everything works as expected.

Task 5. *Create procedure for block graphics.* Now you have made a graphical program, but no mole is yet visible. It is time to create the coordinate system in the blockmole's block world.

a) Make sure you can explain how coordinates in a `PixelWindow` are interpreted, by drawing a simple sketch on paper showing roughly where the positions $(0,0)$, $(300,0)$, $(0,300)$ and $(300,300)$ are in a window that is 300 pixels wide and 500 pixels high. Use figure 4.1 to explain the relationship between underlying window coordinates and block coordinates. Note that the y-axis points downward. ✎

b) Coordinates in `BlockWindow` shall have square, *large* pixels that consist of many window pixels. We call these large pixels *blocks* to distinguish them from the one-pixel image points in `PixelWindow`.
In the block coordinate system for `BlockWindow`, the following applies:

> *The block size* indicates the side length in the square for a block counted in the number of pixels. If the block size is b, then coordinate (x,y) in `BlockWindow` lies at coordinate (bx,by) in `PixelWindow`.

Implement the function `block` in the module `BlockWindow` as below, so that a square is drawn when the procedure is called. The parameter `pos` indicates the block coordinate and the parameter `color` indicates the color. The type-alias declaration of `Pos` gives a descriptive type name for a 2-tuple of integers, which we can use in parameter lists to denote positions in a `BlockWindow`. See the documentation of the `fill` method in

`PixelWindow`. Note that you need to convert the block coordinates in `pos` to window coordinates in `window.fill`. Fill in the missing parts below.

```
type Pos = (Int, Int)

def block(pos: Pos)(color: JColor = JColor.gray): Unit =
  val x = ??? // calculate block's x-coordinate in pixel window
  val y = ??? // calculate block's y-coordinate in pixel window
  window.fill(???)
```

Make sure the code compiles without errors.

c) To test your procedure, call the function `BlockWindow.block` several times in `Main.drawWorld`, both with the default argument omitted and with different colors from the color module. Compile and run your program and verify that everything works as it should.

Task 6. *Create rectangle procedure and underground.* Now you shall write a procedure named `rectangle` that draws a rectangle using the `block` procedure. Then you shall use `rectangle` in `Main.drawWorld` to draw the mole's underground world.

a) Add the procedure `rectangle` in the graphics module. The procedure header should have the following parameters split into three different parameter lists, and return type `Unit`:

```
(leftTop: Pos)(size: (Int, Int))(color: JColor = JColor.gray)
```

The parameter `leftTop` indicates the block coordinate for the rectangle's upper left corner, and `size` indicates `(width, height)` expressed in number of blocks.
Use this nested repetition to draw the rectangle:

```
for y <- ??? do
  for x <- ??? do
    block(x, y)(color)
```

✎ b) In which order are the blocks in the rectangle drawn (vertically or horizontally)? If you are unsure, you can insert a printout of `(x, y)` in the innermost loop to see the order.

c) Add the following code in `Main.drawWorld` so the program draws the underground (that is, lots of soil where the blockmole can dig its tunnels) and some grass.

```
def drawWorld(): Unit =
  BlockWindow.rectangle(0, 0)(size = (30, 4))(Color.grass)
  BlockWindow.rectangle(0, 4)(size = (30, 46))(Color.soil)
```

d) Call `Main.drawWorld` in `Main.run` and test that it works. If any part of the window remains black instead of grass or soil color, check that `block` and `rectangle` are correctly implemented.

Task 7. In `PixelWindow` there are functions to detect keystrokes and mouse clicks. You shall use those functions to control a blockmole. Study the documentation for `awaitEvent` and `Event` in `PixelWindow`, as well as the code in the example program `TestPixelWindow` in the package `introprog.examples`.

a) Add this function in `BlockWindow`:

```
val maxWaitMillis = 10

def waitForKey(): String =
  window.awaitEvent(maxWaitMillis)
  while window.lastEventType != PixelWindow.Event.KeyPressed do
    window.awaitEvent(maxWaitMillis) // skip other events
  println(s"KeyPressed: ${window.lastKey}")
  window.lastKey
```

There are different kinds of events a `PixelWindow` can respond to, like keystrokes and mouse clicks. The function you just added waits for an event in your `PixelWindow` using (`window.awaitEvent`) until a keypress event (`KEY_EVENT`) comes. When a keypress arrives, `window.lastKey` is called to find out which letter or key it was, and this result is also the output of `waitForKey`, since it is the last expression in the block.

b) Extend the procedure `Mole.dig` as below:

```
def dig(): Unit =
  var x = BlockWindow.windowSize._1 / 2
  var y = BlockWindow.windowSize._2 / 2
  var quit = false
  while !quit do
    BlockWindow.block(x, y)(Color.mole)
    val key = BlockWindow.waitForKey()
    if      key == "w" then ???
    else if key == "a" then ???
    else if key == "s" then ???
    else if key == "d" then ???
    else if key == "q" then quit = true
  end while
```

c) Fill in all `???` so that `'w'` moves the mole one step upward, `'a'` one step to the left, `'s'` one step downward and `'d'` one step to the right.

d) Check that `main` only contains two calls: one to `drawWorld` and one to `dig`. Compile and run your program to see if the program responds to keys w, a, s and d.

e) If the program works, there will be many moles forming a long worm, which is a bit odd. Add a call in `Mole.dig` that draws a piece of tunnel at position (x, y) after the call to `BlockWindow.waitForKey` but before the **`if`**-statements. Compile and run your program to dig tunnels with your blockmole.

4.3 Control questions

Review the theory of this week and be prepared to answer these questions when it's your turn to report what you have done during the lab:

1. How do the mole's coordinates change when it moves upward on the screen?
2. How are colors represented with RGB?
3. What is a tuple and how are tuples used in this lab?
4. What does dot notation mean?

5. Give an example of using `import` and explain what happens.
6. What is the advantage of shadowing and local names?
7. We used several single objects as so-called `modules` in this lab. What is the advantage of dividing the code into modules?
8. Go through the goals of the lab and check that you have met them.

4.4 Optional extra tasks

Task 1. The mole can currently dig outside the window. Add some `if` statements at the start of the `while` loop which detects if `x` or `y` is outside the window's edge and move the mole back just inside the edge if so.

Task 2. Moles are not so interested in life above ground, but it might be nice to see how far down the mole has dug. Add a sky color in the object `Color` and draw sky above the grass in `Mole.drawWorld`. Also adjust what you did in the previous task so that the mole stays on the ground. *Tip:* You can benefit from an interactive color picker that you get by calling `introprog.Dialog.selectColor()` in Scala REPL.

Task 3. Change so that the mole does not leave a tunnel behind it when it runs on grass.

Task 4. Change your `rectangle` method so that it draws the same rectangle but *without* using nested loops. This can be accomplished by a call to `PixelWindow.fill`.

Task 5. Let the mole keep digging even if no key is pressed. Keystroke shall change the direction.

a) Create a new method `BlockWindow.waitForKeyNonBlocking` that allows non-blocking keyboard reading as below:

```
def waitForKeyNonBlocking(): String  =
  import PixelWindow.Event.{KeyPressed, Undefined}

  window.awaitEvent(maxWaitMillis)
  while
    window.lastEventType != KeyPressed  &&
    window.lastEventType != Undefined)
  do window.awaitEvent(maxWaitMillis)
  if window.lastEventType == KeyPressed then window.lastKey else ""
```

b) Add a new method `BlockWindow.delay` that should make it possible to slow down the blockmole so it does not run too fast:

```
def delay(millis: Int): Unit = Thread.sleep(millis)
```

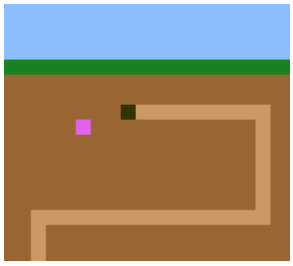
c) Create a new method `Mole.keepOnDigging` that initially is a copy of the method `dig`. Make the following additions/changes:

1. Add two variables `var dx` and `var dy` at the beginning, which keep track of the direction the blockmole digs. Initialize them to `0` and `1` respectively.
2. Add a delay of 200 milliseconds in the infinite loop. Declare a constant `delayMillis` at an appropriate place in `Mole` and use this constant as argument to `delay`.

3. Call `waitForKeyNonBlocking` instead of `waitForKey` and check for key presses like in the code skeleton below. Fill in the missing parts so that the blockmole moves one step in the correct direction each loop iteration.

```
      if      key == "w" then { dy = -1; dx = 0 }
      else if key == "a" then { ??? }
      else if key == "s" then { ??? }
      else if key == "d" then { ??? }
      else if key == "q" then { quit = true }
      y += ???
      x += ???
```

Task 6. *Catch the blockworm.*



Blockworm (*Lumbricus quadratus*) is a fantasy creature in the earthworm family. Known for its ability to teleport instantly from one place to another, making it hard to catch. It has the same RGB color (225, 100, 235) as the real earthworm (*Lumbricus terrestris*) but is square and exactly one block in size. The blockworm is a coveted prey among blockmoles.

a) Add the module `Worm` below in your code and use the procedures in `keepOnDigging` so the blockmole gets a blockworm to chase.

```
object Worm:
  import BlockWindow.Pos

  def nextRandomPos(): Pos =
    import scala.util.Random.nextInt
    val x = nextInt(BlockWindow.windowSize._1)
    val y = nextInt(BlockWindow.windowSize._2 - 7) + 7
    (x, y)

  var pos = nextRandomPos()

  def isHere(p: Pos): Boolean = pos == p

  def draw(): Unit  = BlockWindow.block(pos)(Color.worm)

  def erase(): Unit = BlockWindow.block(pos)(Color.soil)

  val teleportProbability = 0.02

  def randomTeleport(notHere: Pos): Unit =
    if math.random() < Worm.teleportProbability then
      erase()
      while
        pos = nextRandomPos()
        pos == notHere
      do ()
```

```
        draw()

end Worm
```

b) The code in `Worm` assumes that sky exists in the top 7 blocks of the window. How many blocks are sky can ideally be a constant with a good name in a good place. This constant should be used also in `drawWorld`. Fix that!

c) Make the text `"WORM CAUGHT!"` print to the terminal if the blockmole is at the same place as the blockworm.

d) Use the parameter `notHere` to prevent the blockworm from teleporting to the same spot as the blockmole.

e) Make the blockmole get 1000 points each time it catches the blockworm.

f) Make the game last a certain reasonable amount of time before `Game Over`. Use `System.currentTimeMillis` which gives the current number of milliseconds since January 1, 1970. When the game ends, print out the total points the blockmole has accumulated to the terminal.

g) Make the game speed up (i.e. the delay in the game loop decreases) after some time. Concurrently, the probability that the blockworm teleports also increases.

Chapter 5

Lab: `blockbattle0`

Preparations

- ☐ Do exercise `Excercise05` in section 5..
- ☐ You have two weeks to complete `blockbattle`. Already now, read through all exercises in section 6, but first do the basic exercises before starting the lab, especially exercise 6 in section 5..

Chapter 6

Lab: `blockbattle1`

Goals

- ☐ Be able to explain differences and similarities between a single object and objects that are instances of classes.
- ☐ Be able to explain the difference between mutable and immutable objects.
- ☐ Be able to define and instantiate classes and case classes, as well as be able to describe when a case class is most suitable and give some examples of what it offers beyond a regular class.
- ☐ Be able to create and use classes whose instances contain references to other instances (aggregation).
- ☐ Understand the meaning of the instance reference `this`.
- ☐ Be able to create simple match expressions.

Preparations

- ☐ Do exercise `Excercise05` in section 5., especially exercise 6.
- ☐ Do exercise `Excercise06` in section 6..
- ☐ Read through the entire lab and plan your work.
- ☐ Retrieve given code via the course GitHub location.

6.1 Background

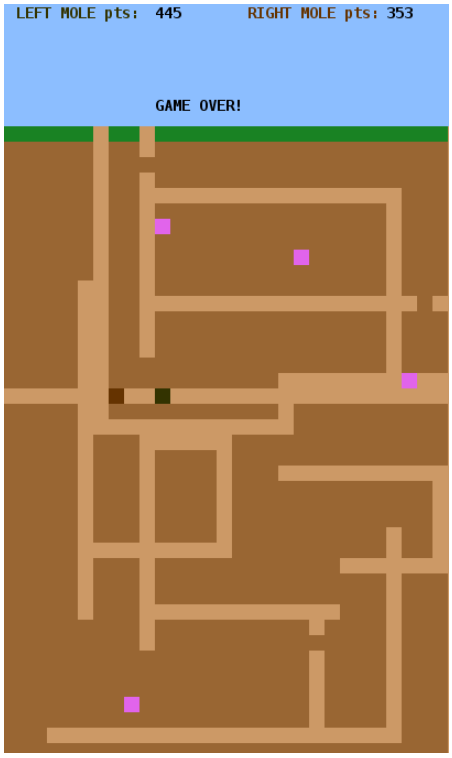


Figure 6.1: A duel over blockworms between two Lund-style block moles captured in an intense digging moment.

In this lab you will practice declaring classes and creating multiple instances of the same class. You will also practice building a larger program from scratch. You will develop a game for two players who share the same keyboard, where the left player controls a block mole with the keys A,S,D,W, and the right player controls another block mole with the arrow keys.

In the picture on the left you can see how the game can look. There is a light brown and a dark brown mole. The scoring is shown at the top in the sky. There are four pink blockworms (see exercise 6 in lab `blockmole`) that the moles compete to catch. When a blockworm teleports to a new random position, it leaves soil behind. When a mole digs up to the grass surface, holes appear in the grass. It gives points to dig tunnels and to catch blockworms.

You decide how scoring should work and what the criteria for when the game ends are etc.

6.2 Mandatory requirements

The following functional requirements must be fulfilled by your program:

- ☐ Each mole moves in its current direction until the user changes direction by pressing ”its” corresponding key, e.g. W or arrow-up.
- ☐ When a mole moves in dark brown soil, light brown tunnels are dug.
- ☐ When a block mole reaches the window edge or the sky, its direction should be reversed.
- ☐ Digging tunnels should yield points.
- ☐ Each player’s points should be displayed during the game.
- ☐ A game should end and *Game over* be displayed when any chosen criterion has been met.

Your code should be designed according to these design requirements:

- ☐ A `Game` is created in the main program with the method `start` which runs the game.
- ☐ Constants should be named and placed in an appropriate companion object.
- ☐ Each class with any corresponding companion object should be in its own source file and belong to the package `blockbattle`.
- ☐ You should start from the classes you implemented in task 6 in exercise `Excercise05`.
- ☐ The class `BlockWindow` converts to internal window coordinates. Other classes

should use block coordinates.

6.3 Optional requirements – choose at least one

You should implement at least one (preferably several) of these requirements:

- ☐ There should be an appropriate number of blockworms (see lab `blockmole` ex. 6, p. 24).
- ☐ The block moles should also have an attribute representing health, e.g. a numeric value between 0 and 100. Health should be weakened slightly when digging tunnels. Health should be visible in the game window, e.g. as a sequence of red blocks in the sky indicating the proportion of max health for each player.
- ☐ Running on grass should affect points and/or health.
- ☐ Catching blockworms should affect points and/or health.
- ☐ There should be yellow block diamonds that give many points if caught first.
- ☐ At game start it should be possible to choose a name for each block mole and the name should appear in the game next to the scoring.
- ☐ It should be faster to move in tunnels compared to digging in soil.
- ☐ If a block mole catches a blockworm, its digging speed should increase.
- ☐ If a block mole collides with another block mole something should happen, e.g. its direction reverses.
- ☐ Show *highscore* at *Game Over*. Highscore is saved with `introprog.IO` in a file which is created if it does not exist, otherwise it is read at startup if available and updated when needed. Save the entire highscore list or just the highest score so far.

6.4 Preparations for presentation

✓ 👁 Before you present your implementation you should be able to orally explain the following:

- ☐ Study someone else's game and give your peer at least one tip on how the readability of their code can be improved. Write down your tips and describe them at the presentation.
- ☐ Describe what measures you have taken to make your code easy to read and understand.
- ☐ Describe how you have gradually developed your program from simpler to more advanced functionality, as well as bugs you have discovered and fixed.
- ☐ Describe which optional requirements your implementation fulfills.
- ☐ Describe how you would need to change the class `Mole` so that it is possible to write

```
new Mole().move().move().reverseDir().move()
```

6.5 Tips and suggestions

1. **Many small steps.** Run compilation with change monitoring using `--watch` in a separate terminal window, so that at every change you can fix any compilation errors. Run and test your program in another terminal window.

2. **Choose good names**. Your code becomes easier to read and modify if you come up with good names for members and place them appropriately. For example, you can

gather global game constants in the companion object of the class `Game`. You can build upon the code below and add members as you discover they are needed. Below is, for example, a function that gives the background color for a given y-coordinate, which is useful when you need to restore the background after a mole has moved.

```
package blockbattle

object Game:
  val windowSize = (30, 50)
  val windowTitle = "EPIC BLOCK BATTLE"
  val blockSize = 14
  val skyRange    = 0 to 7
  val grassRange  = 8 to 8
  object Color { ??? }
  /** Used with the different ranges and eraseBlocks */
  def backgroundColorAtDepth(y: Int): java.awt.Color = ???

class Game(
  val leftPlayerName: String  = "LEFT",
  val rightPlayerName: String = "RIGHT"
):
  import Game.* //direct access to members of companion object

  val window    = new BlockWindow(windowSize, windowTitle, blockSize)
  val leftMole: Mole  = ???
  val rightMole: Mole = ???

  def drawWorld(): Unit = ???

  /** Use to erase old points, e.g updated score */
  def eraseBlocks(x1: Int, y1: Int, x2: Int, y2: Int): Unit = ???

  def update(mole: Mole): Unit = ???  // update, draw new, erase old

  def gameLoop(): Unit = ???

  def start(): Unit =
    println("Start digging!")
    println(s"$leftPlayerName ${leftMole.keyControl}")
    println(s"$rightPlayerName ${rightMole.keyControl}")
    drawWorld()
    gameLoop()
```

3. **Divide your code into functions.** Your code becomes easier to read and modify if you split it into many small functions with good names. In the `Game` class above there are examples of some useful functions. As you extend your program you can add more functions with names like `showPoints`, `gameOver`, etc.

4. **Consider the overall structure.** The program you will write in this lab is larger than what you have done before. Therefore, it is important to consider the structure of your program, which classes are responsible for what and how they cooperate. Feel free to discuss with an instructor if you are uncertain about how the pieces of code you developed in the previous week's exercise 6, the classes `Pos`, `KeyControl`, `Mole`, and `BlockWindow`, are intended to interact. Be sure to test that the different classes and their methods work on their own.

5. **Design of `gameLoop()`.** In a game a so-called game loop is needed which repeats the code that should run on every new screen frame. Each round in the game loop updates data and draws in the game window, as well as includes a suitable delay. A sketch of a typical game loop is shown below:

```
var quit = false
val delayMillis = 80

def gameLoop(): Unit =
  while !quit do
    val t0 = System.currentTimeMillis
    handleEvents()    // changes direction upon key press etc.
    update(leftMole)  // moves, draws, erases, etc.
    update(rightMole)

    val elapsedMillis = (System.currentTimeMillis - t0).toInt
    Thread.sleep((delayMillis - elapsedMillis) max 0)
  end while
end gameLoop
```

6. **Event handling.** A `BlockWindow`, which you implemented in task 6 in exercise `Excercise05`, can, via the call `nextEvent`, return `KeyPressed(key)` upon a key press and `WindowClosed` when the window is closed. If no event is available it returns `Undefined`. Use a loop to process all events until `Undefined` is encountered, as below:

```
def handleEvents(): Unit =
  var e = window.nextEvent()
  while e != BlockWindow.Event.Undefined do
    e match
      case BlockWindow.Event.KeyPressed(key) =>
        ???  // change direction for respective mole

      case BlockWindow.Event.WindowClosed =>
        ???  // end the game loop

    e = window.nextEvent()
  end while
end handleEvents
```

7. **Flicker-free graphics.** To reduce flicker it is best in each iteration of the game loop (1) only to redraw what has changed in order to minimize the time spent drawing, and (2) when making changes draw the new parts before erasing the old parts. To avoid mole flicker you can "*draw first – erase after*" as in the following.[1]

```
window.setBlock(mole.nextPos, mole.color) // draw new
window.setBlock(mole.pos, Color.tunnel)   // erase old
mole.move()                               // update
```

[1] In game development one typically uses so-called *double buffering* (or even *triple buffering*) to achieve completely flicker-free graphics. However, this is beyond the course scope and is not supported by `PixelWindow`.

Chapter 7

Lab: `shuffle`

Goals

- ☐ Be able to create and use sequence collections.
- ☐ Be able to implement the sequence algorithm SHUFFLE that modifies the contents of an array in place.
- ☐ Be able to record the number of occurrences of different values in a sequence.

Preparations

- ☐ Do exercise `Excercise07` in section 7.
- ☐ Read through the entire lab and ensure that you understand how the SHUFFLE algorithm below works.
- ☐ Retrieve the given code via the course's GitHub location.

7.1 Background

This assignment is about card shuffling. Shuffling cards so that every possible permutation (order in which the cards lie) is equally probable is non-trivial; an unsystematic shuffle leads to a skewed distribution.

Given a good random number generator, it is possible to shuffle a deck by putting all cards in a pile and then repeatedly taking a random card from the pile and placing it on top of the deck until all cards are in the deck. The Fisher-Yates algorithm[1] (also called Knuth-shuffle) works this way. Here we refer to this algorithm as SHUFFLE. It is found in pseudocode below. Note especially that the upper limit for r includes i.

Input : Array xs with n values to be shuffled "in place"
Output : xs updated in place with values rearranged in random order

```
for i ← (n − 1) to 0 do
    draw random number r such that 0 ≤ r ≤ i
    swap xs(i) and xs(r)
end
```

A deck has 52 cards, each with a different rank and suit. The card game poker involves drawing cards and getting certain combinations of cards, so-called "hands"[2]. These are ordered from better to worse; the player with the best hand wins. It is therefore

[1] https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle#The_modern_algorithm
[2] https://sv.wikipedia.org/wiki/Pokerhand

```scala
package poker

case class Card(rank: Card.Rank, suit: Card.Suit):
  lazy val show = s"${rank.toString.head}${Card.suitChars(suit.ordinal)}"

object Card:
  enum Rank:
    case Ace, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, Ten, Jack, Queen, King

  enum Suit:
    case Spades, Hearts, Clubs, Diamonds

  val suitChars = "♠♡♣♢"
end Card
```

Figure 7.1: The fully implemented, immutable case class `Card`.

interesting to know with what probability a certain hand appears when drawn from a shuffled deck.
The common poker hands are, in descending order of value, straight flush, four of a kind, full house, flush, straight, three of a kind, two pair, and pair. These are illustrated in section 7.5. There is also a hand called *royal (straight) flush* which denotes a straight flush with an ace as the highest card, but its probability is so low that in simulation it is unlikely to be encountered within a reasonable time.
During the lab, you will start by completing the unfinished class `Deck` shown in section 7.2, and found in the workspace on GitHub.
The lab instructions in section 7.3 give hints on how you should replace `???` in the given code skeletons with your solutions. With the help of the classes `Test` and `Test` you can test that your implementations work correctly.

7.2 Given code

After your implementations of the methods `full` and `shuffle` work, you should use `Deck` in the singleton object `PokerProbability` to find out probabilities for different poker hands when dealing 5 cards from a well shuffled deck.
To assist you, below are code files, some with unfinished code that you shall complete. All code is in a package named `poker`.[3]

- `Card.scala` in fig. 7.1 on page 36 contains the fully implemented case class `Card` which represents a card and has a concise `toString` with rank and suit.
- `Deck.scala` in fig. 7.2 on page 38 contains the mutable class `Deck`, where you shall implement card shuffling in the method `shuffle`. The companion object has methods to create decks. You shall implement the method `full` that creates a complete deck of 52 cards ordered by rank and suit.
- `Hand.scala` in fig. 7.3 on page 39 contains an immutable case class `Hand` that represents a poker hand and has methods to determine which hand it is. The companion object contains factory methods that can create a new hand from

[3] You can browse the classes here:
https://github.com/lunduniversity/introprog/tree/master/workspace/w07_shuffle/

individual cards or by drawing cards from a deck. You shall implement `tally`, which registers the number of cards of a certain rank in an array indexed by the zero-based `ordinal` integer for each card's `rank`.

- `PokerProbability.scala` in fig. 7.4 on page 40 has a main method that calculates poker probabilities, along with the helper method `register` which you shall implement.

- The singleton object `Test.scala`, which also exists in the week's given code, you shall use to test your implementation of `shuffle` with a deck containing only three cards. Repeated shuffles are performed, and the occurrences of each possible permutation are registered.

7.3 Mandatory tasks

Task 1. Implement the SHUFFLE algorithm.

a) Create your own implementation of the method `shuffle` in the class `Deck`. Follow the given algorithm in section 7.1 carefully. You can use `cards.length` to get the deck length, but you are welcome to use `cards.indices.reverse` instead. Implement and use the method `swap`.

b) Run `testShuffle` in `Test` which checks that the shuffle is evenly distributed by shuffling a deck of three cards and counting how often each possible permutation occurs. You should get a printout with six (3!) percentages that should be nearly equal.

Task 2. Create a complete, ordered deck.

a) Implement the method `full` which creates a 52-card standard deck ordered by suit and rank. Use the `Range` values in the companion object `Card`.

b) Run `testCreate` in `Test` and verify that you get cards of all four suits, and both aces and kings.

Task 3. Complete and test `Hand`.

a) Implement `tally` which should give an indexable sequence with 13 places where each index represents the count of the respective rank. Ace counts as lowest.

b) Test the class `Hand` using `Test`.

Task 4. Determine probabilities for different poker hands.

a) Implement the method `register` in `PokerProbability`. Use `from` and `category` in `Hand` to create and categorize a hand from a deck. Store the frequencies in a local array which you, when completed, convert to a sequence with `to(ArraySeq)`.

b) Run the main program in `PokerProbability`, preferably with fifty million iterations depending on your patience and your computer's speed. You should get approximately these probabilities[4]:

[4] https://en.wikipedia.org/wiki/Poker_probability

```scala
package poker

import scala.collection.immutable.ArraySeq

class Deck private (val initCards: ArraySeq[Card]):
  private var cards: Array[Card] = initCards.toArray

  def reset(): Unit = cards = initCards.toArray

  def apply(i: Int): Card = cards(i)

  def toSeq: ArraySeq[Card] = cards.to(ArraySeq)

  def show: String = cards.map(_.show).mkString(" ")

  def peek(n: Int): ArraySeq[Card] =
    cards.take(n).to(ArraySeq)

  def remove(n: Int): ArraySeq[Card] =
    val init = peek(n)
    cards = cards.drop(n)
    init
  end remove

  def prepend(moreCards: Card*): Unit =
    cards = (moreCards ++ cards).toArray

  /** Swaps cards at position a and b. */
  def swap(a: Int, b: Int): Unit = ???

  /** Randomly reorders the cards in this deck. */
  def shuffle(): Unit = ???

object Deck:
  def apply(cards: Seq[Card]): Deck = new Deck(cards.to(ArraySeq))

  /** Creates a new full Deck with 52 cards in rank and suit order. */
  def full(): Deck = ???
```

Figure 7.2: The unfinished class `Deck` with mutable deck of cards.

```scala
package poker

import scala.collection.immutable.ArraySeq

case class Hand(cards: ArraySeq[Card]):
  import Hand._

  /**
   * Yields a sequence of length 13, with positions 0-12 containing the
   * number of cards of that card's rank's ordinal number (zero based).
   */
  lazy val tally: ArraySeq[Int] = ???

  lazy val ranksSorted: ArraySeq[Int] = cards.map(_.rank.ordinal).sorted

  lazy val isFlush: Boolean = cards.length > 0 && cards.forall(_.suit == cards(0).suit)

  lazy val isStraight: Boolean =
    def isInSeq(xs: ArraySeq[Int]): Boolean =
      xs.length > 1 && (0 to xs.length - 2).forall(i => xs(i) == xs(i + 1) - 1)

    isInSeq(ranksSorted) ||  // special case with ace interpreted as 13:
      (ranksSorted(0) == 0) && isInSeq(ranksSorted.drop(1) :+ 13)

  lazy val isStraightFlush: Boolean = isStraight && isFlush
  lazy val isRoyalFlush:    Boolean = ???
  lazy val isFour:          Boolean = tally.contains(4)
  lazy val isFullHouse:     Boolean = tally.contains(3) && tally.contains(2)
  lazy val isThrees:        Boolean = tally.contains(3)
  lazy val isTwoPair:       Boolean = tally.count(_ == 2) == 2
  lazy val isOnePair:       Boolean = tally.contains(2)

  lazy val category: Category =
    if      /* isRoyalFlush    then Category.RoyalFlush
    else if */ isStraightFlush then Category.StraightFlush
    else if isFour          then Category.Fours
    else if isFullHouse     then Category.FullHouse
    else if isFlush         then Category.Flush
    else if isStraight      then Category.Straight
    else if isThrees        then Category.Threes
    else if isTwoPair       then Category.TwoPair
    else if isOnePair       then Category.OnePair
    else                         Category.HighCard

object Hand:
  def apply(cardSeq: Card*): Hand = new Hand(cardSeq.to(ArraySeq))
  def from(deck: Deck): Hand = Hand(deck.peek(5))
  def removeFrom(deck: Deck): Hand = Hand(deck.remove(5))

  enum Category:
    case
      RoyalFlush, StraightFlush, Fours, FullHouse, Flush,
      Straight, Threes, TwoPair, OnePair, HighCard
end Hand
```

Figure 7.3: The unfinished, immutable class `Hand` that represents a poker hand.

```scala
package poker

import scala.collection.immutable.ArraySeq

object PokerProbability:

  /**
   * For a given number of iterations, shuffles a deck, draws a hand and
   * returns a sequence with the frequency of each hand category.
   * Prints a dot every dotStep iteration
   */
  def register(n: Long, deck: Deck, dotStep: Long = 1e6.toLong): ArraySeq[Int] =
    ???
  end register

  @main def simulate: Unit =
    val defaultIter = 5
    val in = scala.io.StdIn.readLine(s"number of million iterations ($defaultIter): ")
    val n = (in.toIntOption.getOrElse(defaultIter) * 1e6).toLong
    val deck = Deck.full()
    val t0 = System.currentTimeMillis()
    val frequencies = register(n, deck)
    for c <- Hand.Category.values do
        val name = c.toString
        val percentages = frequencies(c.ordinal).toDouble / n * 100
        println(f"$name%16s $percentages%10.6f%%")
    end for
    val secs = (System.currentTimeMillis() - t0)/1000.0
    println:
      f"\n*** Total execution time: $secs%3.2f seconds"
```

Figure 7.4: The unfinished singleton object `PokerProbability` which calculates probabilities for different poker hands.

hand	*probability*
Straight flush (excl. Royal flush)	0.00139%
Four of a kind	0.02401%
Full house	0.1441%
Flush (excl. Royal and Straight flush)	0.1965%
Straight	0.3925%

Task 5. Also simulate the probability of Royal flush (a straight flush with ace as highest card). This requires on the order of 10^8 iterations for an accuracy of 2 significant digits. This can take many minutes on a reasonably fast computer, so it might be time for a break during the simulation...

Task 6. Discuss with the supervisor during the review the advantages of using specific enumerated values with `enum` instead of general integers and strings.

Task 7. Discuss with the supervisor during the review the pros and cons of using `lazy val` instead of `val` or `def` in an immutable class.

7.4 Optional extra tasks

Task 1. Implement an interactive card game, e.g. a simple poker variant. Start with something very simple, for example highest-card-wins, and build on with what you find fun.
You can for example create a method `def compareTo(other: Hand): Comparison` in the case class `Hand` which returns `Comparison.Worse` if `other` is worse, `Comparison.Equal` if the hands are equally good, and `Comparison.Better` if `other` is better. Step by step you can make it possible to compare more and more hands according to the special rules that apply for when different hands are considered better or equal. Read about the rules here: https://en.wikipedia.org/wiki/List_of_poker_hands

7.5 Pictures with examples of different poker hands

Figures 7.5 – 7.13 show pictures of different poker hands.

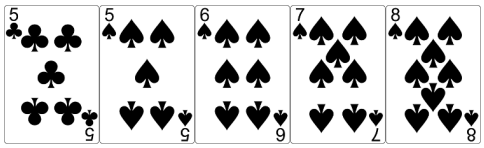


Figure 7.5: Pair: two cards have the same rank.

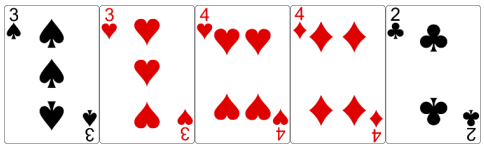


Figure 7.6: Two pair: the hand has two *different* pairs.

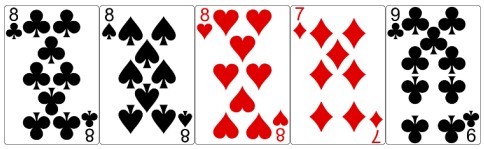


Figure 7.7: Three of a kind: three cards have the same rank.

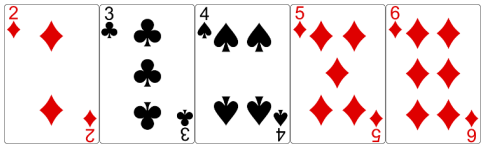


Figure 7.8: Straight: the card ranks form a sequence, ace can be either 1 or 14.

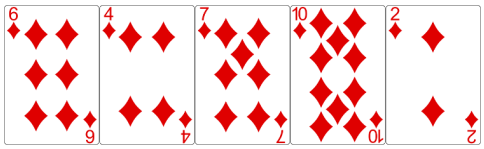


Figure 7.9: Flush: all cards have the same suit.

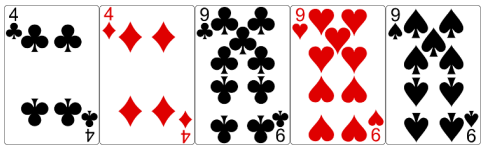


Figure 7.10: Full house: both three of a kind and a pair.

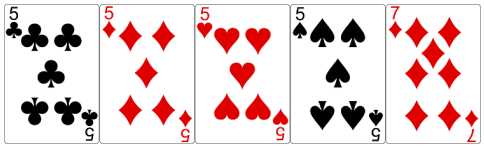


Figure 7.11: Four of a kind: four cards have the same rank.

Figure 7.12: Straight flush: both straight and flush. Special case: If the straight flush's highest card is an ace, it is called *royal flush* or *royal straight flush*.

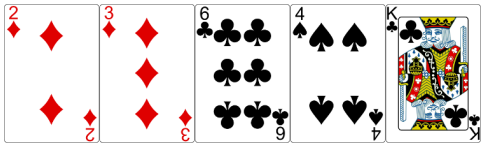


Figure 7.13: High card: no pattern is found.

Part II

Labs and projects of the second study period

Chapter 8

Lab: `life`

Goals

- ☐ Be able to create and use matrices with the help of a generic data type.
- ☐ Be able to iterate over all elements in a matrix.
- ☐ Practice algorithm construction.
- ☐ Practice handling both immutable and mutable objects.
- ☐ Try using a debugger in an integrated development environment (IDE), such as VS Code.

Preparations

- ☐ Do exercise `Excercise08` in chapter 8, especially task 4.
- ☐ Read through the entire lab and study the provided code[1].
- ☐ Read appendix D about debugging.
- ☐ Download the given code via the course's GitHub repository.

[1] https://github.com/lunduniversity/introprog/tree/master/workspace/w08_life

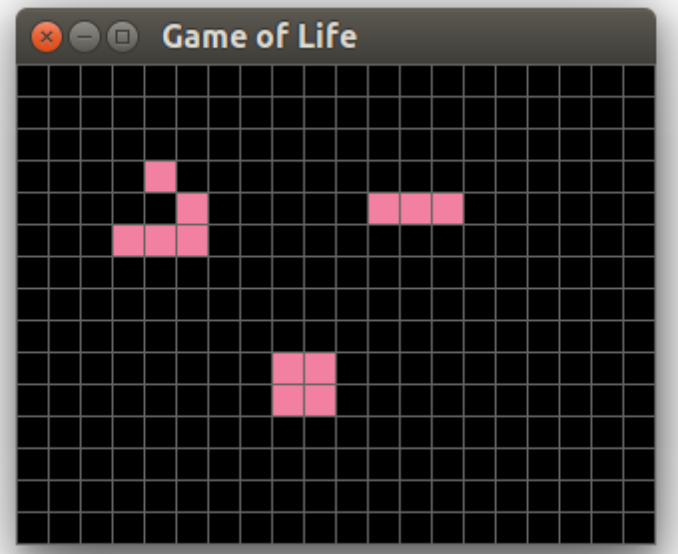


Figure 8.1: A binary, dark data universe of dimension 15×20. The cell colony contains three cell groups: a spaceship of the type *glider*, a *blinker*, and a *block*.

8.1 Background

Game of Life simulates a colony of unicellular organisms that live, reproduce, and die in a matrix, according to some simple but well-chosen rules constructed by the mathematician John Horton Conway in the 1970s. The goal of the game is to simulate many generations from a starting configuration, also called a *cell colony*, where the survival of each individual cell depends on its surroundings. The game has no conscious players, and if the rules are followed, the final result depends solely on the initial configuration.

In *Game of Life*, the universe consists of a matrix with cells that are either alive or dead. Each cell has 8 *neighbors*, consisting of the closest surrounding cells vertically, horizontally, and diagonally. Each cell's state in the next generation is determined by the following rules:

1. **Survival**. If a living cell has two or three neighbors, it survives.
2. **Death**. If a living cell has fewer than two or more than three neighbors, it dies of underpopulation or overpopulation, respectively.
3. **Birth**. If a cell is dead and has exactly three neighbors, it is born and its state changes to living; otherwise it remains dead.

Multiple cell colonies exhibit a "living" behavior where the cell matrix is colonized in interesting ways when a sequence of generations is visualized. This is an example of *emergent* behavior where complex, self-organized structures can arise from simple conditions.

Read more about *Game of Life* on Wikipedia:

- `https://en.wikipedia.org/wiki/Conway's_Game_of_Life`
- `https://sv.wikipedia.org/wiki/Game_of_Life`

8.2 Mandatory requirements

The following functional requirements must be fulfilled by your program:

- ☐ Living cells shall have the beautiful pink[2] RGB color `(242, 128, 161)`.
- ☐ Dead cells shall be black as space.
- ☐ This dark universe with binary data cells shall be drawn in a grid consisting of thin, stylish lines, as shown in fig. 8.1.
- ☐ Keystrokes and mouse clicks shall function according to the following help text, which shall be printed when the program starts:

```
val help = """
  Welcome to GAME OF LIFE!

  Click on cell to toggle.
  Press ENTER for next generation.
  Press SPACE to toggle play/stop.
  Press R to create random life.
  Press BACKSPACE to clear life.
  Close window to exit.
"""
```

  When *play* is activated with the space key, a continuous simulation of the universe shall continue where each new generation is visualized with an appropriate delay in between generations, until the simulation is stopped, e.g. by pressing the space key again. At each *Enter* press, one subsequent generation is shown and any ongoing simulation is stopped. At mouse click on a cell, the life state shall be toggled from live to dead or vice versa. Pressing R shall produce random life. Pressing the backspace key shall render all universe cells dead.

Your code shall be designed according to these design requirements:

- ☐ All classes and singleton objects shall be in the package `life`.
- ☐ There shall be an immutable case class `Life` which represents a cell universe using a `Matrix[Boolean]` from task 4 in this week's exercise.
- ☐ There shall be a class `LifeWindow` that visualizes an instance of the class `Life` in an `introprog.PixelWindow` as in fig. 8.1.

8.3 Optional requirements – choose at least one

You shall implement at least one (preferably several) of these requirements:

- ☐ Cells shall be colored in different colors according to the rules for the next generation. Survival shall still be beautiful pink and persistent death black. The following colors are suggested but you may choose others if you think they look nicer:

```
val UnderPopulated = java.awt.Color.cyan  // a toxic color
val OverPopulated  = java.awt.Color.red   // red squeezed by crowding
val WillBeBorn     = new java.awt.Color(40, 0, 0)  // soon alive
```

  Also give `LifeWindow` a class parameter `isMultiColor` that allows choosing whether cells shall be multicolored or just pink and black as in the basic requirements.

[2] https://www.dsek.se/aktiva/grafiskprofil/farg.php

- ☐ If you press `S` for *Save*, `introprog.Dialog.file("Save Life")` shall be shown and, if the user does not press **Cancel**, the current life shall be saved via `introprog.IO.saveString` to a text file using the `toString` method of `Life`.
- ☐ If you press `O` for *Open*, `introprog.Dialog.file("Open Life")` shall be invoked and a new universe read from a text file according to a suitable format. Reading shall be done with `introprog.IO.loadString` and interpreted into a `Life` instance by a method in the companion object with the following header:

```
def fromString(s: String, rowDelim: String="\n", alive: Char='0'): Life
```

  Test with the file `glider-gun.txt` which should have the following content in the first 11 lines and a total of 32 lines where all lines after the eleventh contain empty life:

```
> head -11 glider-gun.txt
----------------------------------------
-------------------------0----------------
-----------------------0-0----------------
-------------00------00------------00-----
------------0---0----00------------00-----
-00--------0-----0---00-------------------
-00--------0---0-00----0-0----------------
-----------0-----0-------0----------------
------------0---0-------------------------
-------------00-------------------------
----------------------------------------
```

- ☐ The universe shall be circular, i.e., neighbors at the edge shall be on the opposite side by wrapping the indexing, using modulo arithmetic. Introduce a class parameter `isWrapped` in `Life` and a variable `wrapped: Boolean` in the companion object `Life` that controls whether factory methods create a circular universe or not, so you can easily configure this. *Tip:* You can benefit greatly from using `java.lang.Math.floorMod` in the `apply` method in `Life`; the `floorMod` method handles negative values correctly, see documentation for the `Math` package in JDK8.
- ☐ Read about variants of `Game of Life` on Wikipedia and implement alternative rules which can be selected via configuration through the `args` parameter in `main`.
- ☐ Create a class `LifeStatistics` which, through many simulations, shall find the probability that a random cell colony after n generations is still evolving, respectively is completely dead. No visualization with `PixelWindow` shall be done; only the number of cells alive at generation n and the number of cells changed since generation $n-1$ need to be registered.

8.4 Tips and suggestions

1. Here is a suggestion for how you can design the class `Life`:

```
package life

case class Life(cells: Matrix[Boolean]):

  /**Returns true if the cell at location (row, col) is alive, otherwise false.
    * Returns false if the index is outside the bounds of the universe.
```

```
    */
  def apply(row: Int, col: Int): Boolean = ???

  /** Sets the status of the cell at location (row, col) to value. */
  def updated(row: Int, col: Int, value: Boolean): Life = ???

  /** Toggles the status of the cell in place (row, col). */
  def toggled(row: Int, col: Int): Life = ???

  /** Counts the number of living neighbors of the cell in (row, col). */
  def nbrOfNeighbours(row: Int, col: Int): Int = ???

  /**Creates a new Life instance with the next generation of the universe.
    * This is done by applying the rule function to the cells.
    */
  def evolved(rule: (Int, Int, Life) => Boolean = Life.defaultRule):Life =
    var nextGeneration = Life.empty(cells.dim)
    cells.foreachIndex( (r,c) =>
      ???
    )
    nextGeneration

  /** Line separated text where 0 is live cell and - is dead cell. */
  override def toString = ???

object Life:
  /** Creating a universe of dead cells. */
  def empty(dim: (Int, Int)): Life = ???

  /** Creates a universe with random life. */
  def random(dim: (Int, Int)): Life = ???

  /** Implements the rules according to Conways Game of Life. */
  def defaultRule(row: Int, col: Int, current: Life): Boolean = ???
```

You will find the method `nbrOfNeighbours` useful when implementing `defaultRule`. When implementing `random`, the method `foreachIndex` in `Matrix[T]` is convenient to use. If you, as in the suggestion above, let `evolved` take the update rule as a function parameter, it becomes easier to configure the rules in effect and thus easier to create variants of *Game of Life* by introducing new rules in the companion object (see one of the optional tasks with further reference to Wikipedia).

2. Here is a suggestion for how you can design the class `LifeWindow`:

```
package life

import introprog.PixelWindow
import introprog.PixelWindow.Event

object LifeWindow:
  val EventMaxWait = 1 // milliseconds
  var NextGenerationDelay = 200 // milliseconds
  // add more useful constants here, e.g. colors etc.

class LifeWindow(rows: Int, cols: Int):
  import LifeWindow.* // import names from companion object
```

```
var life = Life.empty(rows, cols)
val window: PixelWindow = ???
var quit = false
var play = false

def drawGrid(): Unit = ???

def drawCell(row: Int, col: Int): Unit = ???

def update(newLife: Life): Unit =
  val oldLife = life
  life = newLife
  life.cells.foreachIndex{ ??? }

def handleKey(key: String): Unit = ???

def handleClick(pos: (Int, Int)): Unit = ???

def loopUntilQuit(): Unit = while !quit do
  val t0 = System.currentTimeMillis
  if play then update(life.evolved())
  window.awaitEvent(EventMaxWait)
  while window.lastEventType != PixelWindow.Event.Undefined do
    window.lastEventType match
      case Event.KeyPressed  =>  handleKey(window.lastKey)
      case Event.MousePressed => handleClick(window.lastMousePos)
      case Event.WindowClosed => quit = true
      case _ =>
    window.awaitEvent(EventMaxWait)
  val elapsed = System.currentTimeMillis - t0
  Thread.sleep((NextGenerationDelay - elapsed) max 0)

def start(): Unit = { drawGrid(); loopUntilQuit() }
```

3. **Test your partial solutions thoroughly**. You avoid hard subsequent errors if you test your partial solutions carefully before proceeding. For example, it is common to confuse rows and columns, but you will discover this if you test all relevant functions with a rectangular universe where the number of rows and columns differ.

4. **Take advantage of your IDE**. There are many useful features in an integrated development environment (IDE) such as Microsoft VS Code[3] with the Metals extension[4]. Read online about your IDE and learn about things you didn't know that seem useful. Especially look up the list of keyboard shortcuts[5] and learn some shortcuts that can help you speed up common tasks.

5. Study the documentation about the debugger in your IDE.[6] [7]

[3] https://code.visualstudio.com/
[4] https://scalameta.org/metals/docs/editors/vscode
[5] https://code.visualstudio.com/docs/getstarted/keybindings
[6] https://scalameta.org/metals/docs/editors/vscode#running-and-debugging-your-code
[7] https://code.visualstudio.com/docs/editor/debugging

Chapter 9

Lab: `words`

Goals

- ☐ Be able to use key-value tables with the collection `Map`.
- ☐ Be able to use sets with the collection `Set`.
- ☐ Understand the differences and similarities between a sequence and a set.
- ☐ Understand the differences and similarities between a sequence of pairs and a key-value table.
- ☐ Be able to implement algorithms that use nested structures.
- ☐ Be able to combine mutable and immutable collections.

Preparations

- ☐ Do exercise `Excercise09` in section 9.
- ☐ Read through the entire laboratory assignment.
- ☐ Fetch and read the given code from the course's GitHub page.

9.1 Background

This assignment deals with analysis of natural language (eng. *Natural Language Processing, NLP*). Language analysis often relies on statistics about the occurrence of different words in long texts. You will write code that, based on a long text, for example a book, can help you answer this type of questions:

- How common is a certain word in a given text?
- What is the most common word that follows a certain word?
- How can one generate word sequences that resemble the word order in a given text?

To be able to answer such questions, you will create frequency tables and also so called *n-grams*; sequences of n words that occur consecutively in a text. Examples of some 2-grams (also called *bigrams*) found in the previous sentence: (för, att), (att, kunna), (kunna, svara), (svara, på), (på, sådana), and so on.[1]

[1] You can investigate various n-grams in a large collection of books using Google's n-gram viewer: `https://books.google.com/ngrams/`

9.2 Mandatory tasks

You shall build your program with an editor, e.g. VS code, and compile and run your code in the terminal using `scala-cli` in *watch mode* with the workflow described in appendix F section F.2.1. While you develop your program step by step, you should simultaneously experiment in the REPL to explore how to use collection methods to solve the tasks. Code to start from is here: https://github.com/lunduniversity/introprog/tree/master/workspace/w09_words
These incomplete code files are in the package `nlp`:

- `FreqMapBuilder.scala` contains a skeleton for a class to, word by word, build a key-value table that records the number of occurrences of different words. Implementing this was part of the exercise you did earlier this week.
- `Text.scala` contains a skeleton for a class that can do text processing by analyzing words in a text.
- `Main.scala` contains a partial main program that you can use in the later part of the lab.

Task 1. *Create frequency tables.* You shall use `FreqMapBuilder` from this week's exercise to create frequency tables of type `Map[String, Int]`, where the key-value pairs in the table denote the number of occurrences of a certain string.

a) Put the class `FreqMapBuilder` in a package called `nlp` and compile.

```
package nlp

class FreqMapBuilder:
  private val register = collection.mutable.Map.empty[String, Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit = ???

object FreqMapBuilder:
  /** Create new FreqMapBuilder and count the strings in xs*/
  def apply(xs: String*): FreqMapBuilder = ???
```

b) Test carefully that your `FreqMapBuilder` works correctly. Example test in REPL:

```
scala> import nlp.*

scala> val fmb = FreqMapBuilder("hej", "on", "dough")
fmb: nlp.FreqMapBuilder = nlp.FreqMapBuilder@458f85ef

scala> fmb.add("hej")

scala> fmb.toMap
res0: Map[String,Int] = Map(on -> 1, hej -> 2, dough -> 1)

scala> (1 to Short.MaxValue).foreach(i => fmb.add(i.toString))

scala> fmb.toMap.size
res1: Int = 32770

```

```
scala> fmb.toMap
res2: Map[String,Int] =
  Map(10292 -> 1, 19125 -> 1, 26985 -> 1, 29301 -> 1, 5451 -> 1, 4018 -> 1, 31
```

In the upcoming tasks you shall step by step create and test the case class `Text`.

```scala
package nlp

case class Text(source: String):
  lazy val words: Vector[String] = ???  // dela upp source i ord

  lazy val distinct: Vector[String] = words.distinct

  lazy val wordSet: Set[String] = words.toSet

  lazy val wordsOfLength: Map[Int, Set[String]] = wordSet.groupBy(_.length)

  lazy val wordFreq: Map[String, Int] = ???  // use FreqMapBuilder

  def ngrams(n: Int): Vector[Vector[String]] = ???  // use sliding

  lazy val bigrams: Vector[(String, String)] =
    ngrams(2).map(xs => (xs(0), xs(1)))

  lazy val followFreq: Map[String, Map[String, Int]] = ??? //nested map

  lazy val follows: Map[String, String] =
    followFreq.map( (key, followMap) =>
      val maxByFreq: (String, Int) = followMap.maxBy(_._2)
      val mostCommonFollower: String = maxByFreq._1
      (key, mostCommonFollower)
    )
    // or shorter with the same result: (easier or harder to read)
    //  followFreq.map((k, v) => k -> v.maxBy(_._2)._1)

object Text:
  def fromFile(fileName: String, encoding: String = "UTF-8"): Text =
    val source = scala.io.Source.fromFile(fileName, encoding)
    val txt = try source.mkString finally source.close()
    Text(txt)

  def fromURL(url: String, encoding: String = "UTF-8"): Text =
    val source = scala.io.Source.fromURL(url, encoding)
    val txt = try source.mkString finally source.close()
    Text(txt)
```

Task 2. *Split a string into words.* You shall implement the member `words`. It should contain a vector with all words in `source`, without any other characters than letters. You achieve this by starting from the string `source` and sequentially doing the following:

1. Replace all characters in `source` for which `isLetter` is false with `' '`

2. Split the string from the previous step into an array of strings with `split(' ')`
3. Filter out all empty strings
4. Convert all letters in all strings to lowercase
5. Convert the array into a sequence of type `Vector[String]`.

Test that `words` and the values using `words` work in REPL:

```
scala> val t = Text("Cucumber is not tomato, but cucumber is a vegetable.")

scala> t.words
res1: Vector[String] =
  Vector(cucumber, is, not, tomato, but, cucumber, is, a, vegetable)

scala> t.distinct
res2: Vector[String] =
  Vector(cucumber, is, no, tomato, but, a, vegetable)

scala> t.wordSet
res3: Set[String] = Set(cucumber, is, no, tomato, but, a, vegetable)

scala> t.wordsOfLength(3)
res4: Set[String] = Set(but)
```

Task 3. You shall now create the word frequency table `wordFreq` by registering word occurrences using `FreqMapBuilder`. The table `wordFreq` shall consist of key-value pairs `w -> f` where `f` is the number of times the word `w` occurs in `words`. Test `wordFreq` by downloading the book ”Treasure Island” written by Robert Louis Stevenson[2] and check the frequency for various common words. Which word is the most common? The second most common?

```
scala> val piratbok = Text.fromURL("https://fileadmin.cs.lth.se/pgk/skattkammaron.txt")
val piratbok: nlp.Text = Text(Herr Trelawney, doktor Livesey and the other gentlemen ha

scala> piratbok.words.size
val res0: Int = 69438

scala> piratbok.wordFreq("pirat")
val res1: Int = 7
```

Links to UTF-8 formatted books you can use in your tests:

- ”Treasure Island” by R. L. Stevenson:
  https://fileadmin.cs.lth.se/pgk/skattkammaron.txt
- ”Inferno” by August Strindberg:
  https://fileadmin.cs.lth.se/pgk/inferno.txt
- ”Pride and Prejudice” by Jane Austen:
  https://fileadmin.cs.lth.se/pgk/prideandprejudice.txt
- Project Gutenberg with many freely available books in text format:
  https://www.gutenberg.org/

[2] Copyright for this book has expired, so you are not infringing copyright (in a legal sense).

Task 4. Implement the method `ngrams` that gives a sequence with all word sequences of length n. *Tip:* In this week's exercise, you examined how the method `sliding` works, which you can use to create n-grams. Use `toVector` on the result from `sliding`. Test carefully that `ngrams` and `bigrams` work correctly before proceeding.

```
scala> piratbok.ngrams(3).take(2)
val res1: Vector[Vector[String]] =
  Vector(Vector(herr, trelawney, doktor), Vector(trelawney, doktor, livesey))

scala> piratbok.bigrams.take(2)
val res2: Vector[(String, String)] =
  Vector((herr,trelawney), (trelawney,doktor))
```

Task 5. Implement `followFreq`, which shall contain a key-value table where the value in turn is a frequency table over words that follow the key.
By analyzing all word pairs we can find out which word is the most common to follow a given word. The method `bigrams` gives us all word pairs `(w1, w2)` where `w2` follows `w1`. We can save statistics over following words in a key-value table with mappings `w -> f` where the key `w` is a word and the value `f` is a frequency table of type `Map[String, Int]`. In the frequency table, we store the frequency for all words that follow `w`. You shall thus build a nested table of type `Map[String, Map[String, Int]]`.
✎ Draw a picture of the nested structure.
Implement the method followFreq based on the pseudocode below:

```
val result = collection.mutable.Map.empty[String, FreqMapBuilder]
for (key, next) <- bigrams do
  if /* key already defined in result */ then
    /* at "location" result(key): add next to the frequency table */
  else
    /* add (key -> new frequency table with next) to result */
end for
result.map(p => p._1 -> p._2.toMap).toMap // toMap gives immutable Map
```

✎ Make printouts to find out answers to the following questions. Write down your answers and be ready to present them during the control questions (see section 9.3).

a) Which words usually follow *han* and *hon* in Stevenson's "Treasure Island"?

b) Which words usually follow *han* and *hon* in Strindberg's "Inferno"?

c) Which words usually follow *he* and *she* in Austen's "Pride and Prejudice"?

Task 6. Create a main program that reports any interesting measures about the words in a text. The program should take the text source as an argument, given as a URL or a filename. Write the main program in the file `Main.scala` in a single object named `Main`. An example of a report your main program can generate is shown below. Here is also an integer argument controlling the length of the top lists.

```
> scala run . -- https://fileadmin.cs.lth.se/pgk/skattkammaron.txt 13

Source: https://fileadmin.cs.lth.se/pgk/skattkammaron.txt

*** Number of words: 69438

*** The 13 most common words and their frequency:
```

```
(och,3089), (jag,2007), (att,1594), (det,1382), (en,1262),
(i,1244), (som,1132), (på,1068), (han,1063), (var,990),
(med,854), (den,774), (av,740)

*** The 13 longest words and their length:
(besättningsmedlemmarnas,23), (befästningsanordningar,22),
(temperamentsuppvisning,22), (undsättningsexpedition,22),
(besättningsmedlemmarna,22), (försiktighetsåtgärder,21),
(undsättningsfartyget,20), (sjukdomsframkallande,20),
(husföreståndarinnans,20), (sjömansterminologin,19),
(parlamentärsflaggan,19), (bregravningsplatsen,19),
(tidvattenströmmarna,19)
```

Example main program that can produce the above output:

```
package nlp

object Main:
  val defaultUrl = "https://fileadmin.cs.lth.se/pgk/skattkammaron.txt"
  val defaultN = 10

  def mostFrequentWords(n: Int, freqMap: Map[String, Int]): Vector[(String, Int)] = ???


/*  |Källa: $from
    |
    |*** Antal ord: ${text.words.size}
    |
    |*** De $n vanligaste orden och deras frekvens:
    |${mostFrequentWords(n, text.wordFreq).mkString(", ")}
    |
    |*** De $n längsta orden och deras längd:
    |$longestWordsWithLength
*/

  def report(text: Text, from: String, n: Int): String =
    val longestWordsWithLength =
      mostFrequentWords(n, text.distinct.map(w => (w, w.length)).toMap).mkString(", ")
    s"""
    |Source: $from
    |
    |*** Number of words: ${text.words.size}
    |
    |*** The $n most common words and their frequency:
    |${mostFrequentWords(n, text.wordFreq).mkString(", ")}
    |
    |*** The $n longest words and their length:
    """.stripMargin

  def main(args: Array[String]): Unit =
    val location = if args.isEmpty then defaultUrl else args(0)
    val n = if args.length < 2 then defaultN else args(1).toInt
    val text =
      if location.startsWith("http") then Text.fromURL(location)
      else Text.fromFile(location)

```

```
    println(report(text, location, n))
```

Task 7. Pair up with another student and plan how you together can, using https://fileadmin.cs.lth.se/pgk/muntabot, train for the oral exam where you mutually act as "mock examiners". Make a plan for when you will test each other on which weeks. Show your plan to the instructor and discuss what it means to be a good "mock examiner".

9.3 Control questions

1. What are your answers to task 5 a) b) c) on page 55?
2. In what order do elements appear when calling `distinct` on a sequence?
3. If you iterate over a set, in what order are the elements processed?
4. Give an example of when it is appropriate to use a set instead of a sequence of distinct values.
5. Are all keys in a key-value table guaranteed to be unique?
6. Are all values in a key-value table guaranteed to be unique?
7. The LTH engineer Oddput Clementin wants to sum the length of each string in a set and writes:

```
scala> Set("hej", "on", "dej").map(_.length).sum
res0: Int = 5
```

   Why does this produce an incorrect result? How can Oddput fix the problem?

9.4 Optional tasks

Task 1. Extend the class `Text` and implement the following method that should return a random word from `wordSet`. Each word should be equally probable.

```
def randomWord: String = ???
```

Task 2. With NLP, you can generate random sentences that statistically resemble "real", human-created sentences.
Implement the method `randomSeq(firstWord, n)` below in the class `Text`. It should produce a sequence $w_1, w_2, ..., w_n$ where w_1 is `firstWord` and w_{i+1} is a random word drawn among the words that follow w_i. You can achieve this by selecting each following word w_{i+1} from `keys.toVector` for the `followFreq` table associated with w_i. The words should be drawn from the following set with equal probability.

```
def randomSeq(firstWord: String, n: Int): Vector[String] = ???
```

Task 3. To make your computer-generated sentences really resemble human language, we can create the most probable sentences of different lengths from our word frequency analysis.
Add the method `mostCommonSeq` in the class `Text` as follows:

```
def mostCommonSeq(firstWord: String, n: Int): Vector[String] = ???
```

a) Implement the method so that the result is a sequence of `n` words. The sequence should start with `firstWord` and then be followed by the word that is the *most common* word following `firstWord`, and thereafter the most common following word after that, etc. *Tip:* Use a local variable `val result` which is an ArrayBuffer to which you add the following words in a `while` loop.

b) Compare the random sequences with sequences generated with `randomSeq` in task 2. Which sequences most resemble "real" sentences?

Task 4. Use existing collection methods instead of `FreqMapBuilder` to register following words.

a) Explore in REPL how the method `groupBy(x => x)` behaves when applied on a collection of strings. Search for and study the documentation for `groupBy`.

b) Introduce the value `lazy val wordFreq2`. It should yield the same result as `wordFreq` but implemented using `groupBy` and `map` instead of `FreqMapBuilder`.

c) Compare performance between `wordFreq2` and `wordFreq`. Which is fastest for large texts? Is there a big difference? ★

d) Introduce the value `lazy val followsFreq2`. It should yield the same result as `followsFreq` but implemented using `groupBy` and `map` instead of `FreqMapBuilder`. This task is rather tricky. Experiment in REPL and build a solution step by step. *Tip:*

```
bigrams
  .groupBy(???)
  .map(p => p._1 -> p._2.map(???).groupBy(???).map(???))
```

e) Compare performance between `followsFreq2` and `followsFreq`. Which is fastest for large texts? Is there a large difference? ★

Task 5. *Make `FreqMapBuilder` generic.* Generic structures, that is those that have type parameters, are often significantly more useful. If you make `FreqMapBuilder` generic by introducing a type parameter instead of hard-coding the type to `String`, you can use `FreqMapBuilder` with arbitrary element types.

a) Examine `FreqMapBuilder` and identify everything in that class that is specific to the type `String`.

b) Introduce a type parameter `A` inside square brackets after the class name and then use `A` instead of `String` in all methods.

c) Test that your generic frequency table builder works on sequences that contain other than strings.
This works because nothing in `FreqMapBuilder` really assumes that the elements counted are of string type (it suffices that there is a proper `equals` and `hashCode`).

Chapter 10

Team lab: `snake0`

Goals

- ☐ Be able to use inheritance.
- ☐ Be able to override members in a supertype.
- ☐ Be able to explain the concept of dynamic binding.
- ☐ Be able to use abstract classes and create a class hierarchy.

Preparations

- ☐ Do exercise `Excercise09` in chapter 10., especially task 4.
- ☐ Read the documentation for `introprog.BlockGame`.
- ☐ Download the provided code via the course GitHub repository.
- ☐ Read through the entire lab and prepare yourself for the first group meeting.
- ☐ Discuss in your collaboration group how you will divide the code between you into several different parts, which you can work on separately. Such a part can be a class, a trait, an object, a package, or a function.
- ☐ Each part should have a **main responsible** individual.
- ☐ The division of work should be roughly evenly distributed among the group members.
- ☐ The person who is the main responsible for a certain part presents that part.
- ☐ You shall develop a common group checklist for code review. Each group member shall review at least one other group member's code according to the checklist.
- ☐ The group collaboration takes place over **two weeks** divided into two partial presentations. At the first presentation, the work plan and ongoing development shall be presented. At the second occasion, the completed solutions shall be presented by the respective main responsible individual.
- ☐ At the first presentation, you shall explain to the supervisor how you have divided the code and who is primarily responsible for what and what your responsibility includes, as well as how you practically work to synchronize your development.
- ☐ The group laboratory work is an extra large assignment and group work requires lead time so that you have enough time to coordinate among yourselves. You therefore need to plan to work on some part of the group lab virtually every day during the available weeks, and be ready to contribute to discussions.
- ☐ Meet in your collaboration group and discuss your working method based on the following questions:
  - Which requirements are you going to implement?
  - How will you work with shared code components?

- How will you share the code components you develop individually? It is educational to try manual merge of changes initially.
  * If you do not use git: Consider when and how you will merge your changes. Use, e.g., a USB drive or a shared storage location on the network that only your group has access to. Communicate when it is time to sync via chat or email.
  * If you use git: How will you use branches? How should your `.gitignore` look?

10.1 Background

The game *Snake*[1] became very popular in Sweden already in the 1980s, often played on the legendary ABC80 computer. The game exists in several variants, both for a single player and as a duel between two players. Each player controls a worm with a head and tail that constantly moves forward. The goal is to avoid running into a worm tail and to collect points by e.g. eating an apple.
Figure 10.1 shows a start dialog where you can select the number of players, as well as examples of a game with one and two players. In the one-player variant, the worm's head approaches the apple and may manage to eat it if the player steers correctly. In the two-player variant, the green worm wins because the blue worm accidentally ran into the green worm's tail.

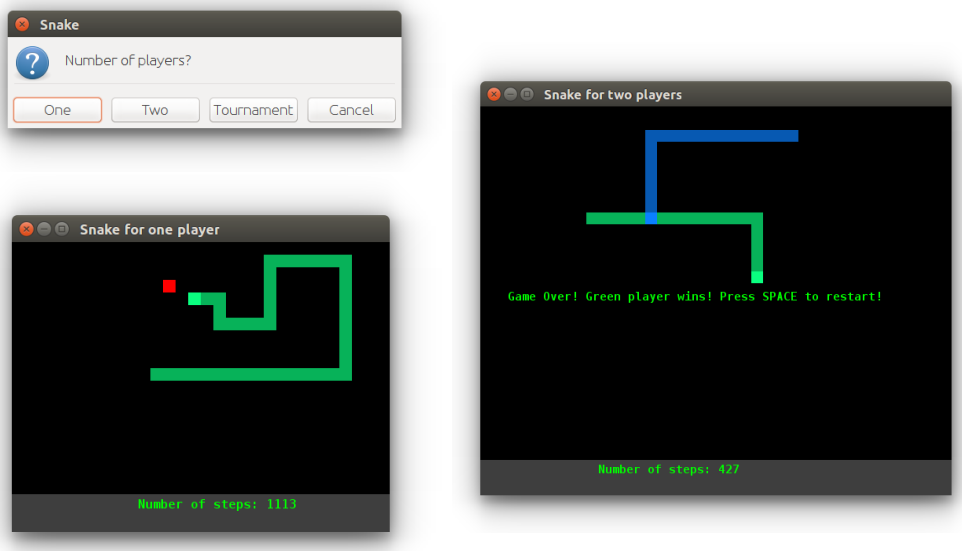


Figure 10.1: The game Snake for one player with apple and for two players without apple.

10.2 Mandatory functional requirements

The following functional requirements must be met by your program if you are six people in the group. If you are fewer, the mandatory requirements are those shown in Table 10.1.

[1] Also called "the worm". https://en.wikipedia.org/wiki/Snake_(video_game)

- ☐ **`Player`**. There must be players that correspond to human users and who have a name and four keys they can play with. Each player has their own snake which they can control with their keys.
- ☐ **`Snake`**. There must be snakes. A snake consists of a number of blocks, where the foremost block is called the head and the rest of the blocks are called the tail. The head has a lighter color than the body. The tail's length grows during the game. A snake moves in a certain direction and each player can change the direction of their snake with their keys, in one of four directions `North`, `South`, `East` or `West`.
- ☐ **`Apple`**. There must be (at least one) apple. An apple consists of a red block and is in a random position. An apple can be eaten by a snake if the snake's head hits the apple. Each time an apple is eaten by a snake, the apple teleports to a new position and can be eaten again.
- ☐ **`Banana`**. There must be (at least one) banana. A banana consists of three vertical yellow blocks and is in a random position. A banana is eaten by a snake if the snake's head hits the banana. Each time a banana is eaten by a snake it teleports to a new random position and can be eaten again.
- ☐ **`Monster`**. There must be (at least one) monster. A monster consists of five pink blocks in a cross formation. A monster spawns at a random position and moves in a direction determined at the monster's birth. A monster will be eaten and die if the snake's head touches any monster block.
- ☐ **`OneplayerGame`**. It must be possible to play alone. In the single-player variant there is one snake and at least one apple (and possibly bananas and monsters as well). Each time the user's snake manages to eat a fruit, the user gets points. When the snake has eaten a certain number of apples, or if the snake has been eaten by a monster, the game ends and the points are shown. A snake's tail should grow at regular time intervals or when it eats fruit.
- ☐ **`TwoplayerGame`**. It must be possible to play two-player. In the two-player variant there are two snakes. There are also apples, bananas, and monsters. If a snake eats a banana, its tail grows. When the snake has eaten a certain number of apples, or if the snake has been eaten by a monster, the game ends and the points are shown. A snake's tail should grow at regular time intervals or when it eats fruit.
- ☐ **`Settings`**. Settings for the game must be configurable through a text file which is loaded at the start of the game. Settings should be a context parameter.

Table 10.1: Requirements that must be implemented at minimum depending on group size. If you have special reasons, you can after approval from the course coordinator do the lab individually.

Requirement / Number of people	1	2	3	4	5	6
`Player`	√	√	√	√	√	√
`OnePlayerGame`	√				√	√
`TwoPlayerGame`		√	√	√	√	√
`Snake`	√	√	√	√	√	√
`Apple`	√		√	√	√	√
`Banana`				√		√
`Monster`			√			√
`Settings`	√	√	√	√	√	√

10.3 Mandatory design requirements

☐ Snake game must be able to start with the main program below. The main program may be changed as needed in accordance with the minimum requirements regarding group size in Table 10.1, as well as optional extra requirements in section 10.4, and your own ideas.

```
package snake

def createOnePlayerGame(): Unit =
  Settings.default.windowTitle = "Snake: One Player"
  OnePlayerGame().play()

def createTwoPlayerGame(): Unit =
  Settings.default.windowTitle = "Snake: Two Player"
  TwoPlayerGame().play()

@main
def run: Unit =
  val buttons =
    Seq("One", "Two", "Competition", "Tournament", "Cancel")
  val selected =
    introprog.Dialog.select("Number of players?", buttons, "Snake")
  selected match
    case "One"         => createOnePlayerGame()
    case "Two"         => createTwoPlayerGame()
    case "Competition" => println(s"TODO: $selected")
    case "Tournament"  => println(s"TODO: $selected")
    case  _            => println("Goodbye!")
```

☐ The game must be built on `introprog.BlockGame` according to the type hierarchy in fig. 10.2.

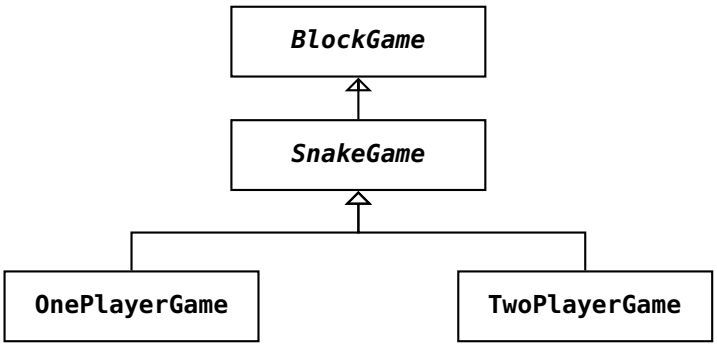


Figure 10.2: Inheritance hierarchy with the class `introprog.BlockGame` as base type.

☐ Snakes, monsters, and fruits must derive from the base type `Entity` according to the type hierarchy in 10.3.

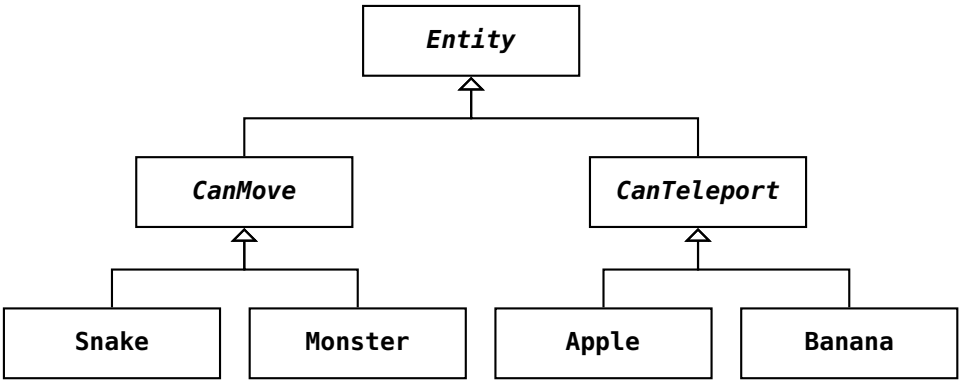


Figure 10.3: Inheritance hierarchy with the class `Entity` as base type.

☐ `Entity` represents a creature in a game and should look as follows:

```
package snake

trait Entity:
  def draw(): Unit

  def erase(): Unit

  def update(): Unit

  def reset(): Unit

  infix def isOccupyingBlockAt(p: Pos): Boolean
```

The methods `draw` and `erase` are called when drawing and erasing respectively. The method `reset` restores the original state. The method `update` is called once in each round in the game loop. The predicate `isOccupyingBlockAt` returns true if the position `p` is among the blocks that the creature occupies on the screen.

☐ `CanMove` represents an entity that can move at a certain speed, as follows:

```
package snake

trait CanMove extends Entity:
  def move(): Unit

  var movesPerSecond: Double = 20.0

  final def millisBetweenMoves: Int =
    (1000 / movesPerSecond).round.toInt max 1

  private var _timestampLastMove: Long = System.currentTimeMillis

  final def timestampLastMove = _timestampLastMove

  override final def update(): Unit =
```

```
    // move if time has passed millisBetweenMoves
    if System.currentTimeMillis >
         _timestampLastMove + millisBetweenMoves
    then
      _timestampLastMove = System.currentTimeMillis
      move()
```

☐ CanTeleport represents an entity that exists in a certain location but after a certain number of updates teleports unpredictably to a new position:

```
package snake

trait CanTeleport extends Entity:
  private var _pos = teleport()

  def pos: Pos = _pos

  protected var nbrOfStepsSinceLastTeleport = 0

  def teleportAfterSteps: Int

  def teleport(): Pos

  def update(): Unit =
    nbrOfStepsSinceLastTeleport += 1
    if nbrOfStepsSinceLastTeleport > teleportAfterSteps
    then reset()

  def reset(): Unit =
    nbrOfStepsSinceLastTeleport = 0
    _pos = teleport()
```

☐ There must be an enumeration State in the singleton object SnakeGame representing the game's overall state as follows:

```
package snake

object SnakeGame:
  enum State:
    case Starting, Playing, GameOver, Quitting
  export State.* // makes all states visible in SnakeGame
```

☐ In each round in the game loop, the following logic shall be executed. This code is preferably placed in gameLoopAction, see further SnakeGame in section 10.5.

```
    if state == Playing && !isPaused then
      _iterationsSinceStart += 1
      entities.foreach(_.erase())
      entities.foreach(_.update())
      entities.foreach(_.draw())
```

```
      onIteration()
      if isGameOver then enterGameOverState()
```

- ☐ There must be a singleton object `Colors` where all colors used in the game are collected.
- ☐ The file `pairs.scala` should, according to the lab preparations in exercise 4 on page 107, contain `Pair[T]`, `Dim`, `Pos`, `Dir`, `North`, `South`, `East`, `West`. See workspace here:
  https://github.com/lunduniversity/introprog/tree/master/workspace/
- ☐ The class `Player` should look as follows:

```
package snake

class Player(
  var name: String,
  var keyMap: Player.KeyMap,
  val snake: Snake,
  var points: Int = 0,    // TODO: count points when e.g. eating apple
):
  def handleKey(key: String): Unit =
    ??? //if key is included in keyMap then update snake.dir

object Player:
  enum KeyMap(left: String, right: String, up: String, down: String):
    val dir = Map(left -> West, right -> East, up -> North, down -> South)
    case Letters extends KeyMap("a", "d", "w", "s")
    case Arrows extends KeyMap("Left", "Right", "Up", "Down")
```

10.4 Optional requirements – each person must choose at least one

Each person in the group must implement *at least one* (preferably several) of the requirements below. Implementing several of these requirements will make the game significantly more fun.

- ☐ **`Points`**. Introduce a points system where the score depends on e.g. the length of the tail, number of steps, number of turns, number of apples eaten, etc.
- ☐ **`Highscore`**. The game should show a list of players who have achieved the highest scores.
- ☐ **`Apple`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`Monster`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`Banana`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`SelfTailCrash`**. If a player runs into their own snake's tail, the game is lost. (If this requirement is not implemented, then the player *may* pass through their own tail without consequence.)

- ☐ **BoundaryCrash**. If a player moves off the game board, the game is lost. (If this requirement is not implemented, the snake should continue on the other side of the board when it reaches the edge.)
- ☐ **EnterPlayerName**. Players can enter their name, e.g. via a dialog or through arguments to `main`. The name is used in the message field during scoring and in messages about who won.
- ☐ **OnePlayerGame**. You can choose to implement `OnePlayerGame` if it is not already part of the mandatory requirements.
- ☐ **TwoPlayerComp extends Competition**. Two players should be able to compete in a best-of-n series of `TwoPlayerGame.play` matches, where the one who wins most matches is the overall winner.
- ☐ **SinglePlayerComp extends Competition**. Several players should be able to compete in single-player Snake, where the player with the highest score over n `OnePlayerGame` games is the overall winner.
- ☐ **Tournament extends Competition**. Many players should be able to play a tournament.[2] Player names are read from a text file. Optional variants:
  - ☐ **KnockOut extends Tournament**. It should be possible to play a knockout tournament, which ends with final after semi-final, etc., depending on the number of players.
  - ☐ **RoundRobin extends Tournament**. It should be possible to play a round-robin tournament where all possible player pairs meet in random order.

10.5 Tips and suggestions

This section presents sketches for some of the classes needed in accordance with the design requirements. It is allowed to change, remove and add, as long as the mandatory design requirements are met. The code can be found here:
https://github.com/lunduniversity/introprog/tree/master/workspace/
Here is a sketch of the `Snake` class:

```
package snake

import java.awt.Color

class Snake (
  val initPos: Pos,
  val initDir: Dir,
  val headColor: Color,
  val tailColor: Color,
 )(using ctx: SnakeGame, settings: Settings) extends CanMove:
  var dir: Dir = initDir
  val initBody: List[Pos] = List(initPos + initDir, initPos)
  val body: scala.collection.mutable.Buffer[Pos] = initBody.toBuffer

  val initLength: Int = settings.snake.initLength
  val growEvery: Int = settings.snake.growEvery
  val startGrowingAfter: Int = settings.snake.startGrowingAfter

  private var _nbrOfSteps = 0
  def nbrOfSteps: Int = _nbrOfSteps
```

[2] https://en.wikipedia.org/wiki/Tournament

```
  private var _nbrOfApples = 0
  def nbrOfApples: Int = _nbrOfApples

  def reset(): Unit = ???  // restore start state, give correct tail length

  def grow(): Unit = ??? // grow in the right direction with extra tail position

  def shrink(): Unit = ??? // shrink the tail if the body length is greater than 2

  def isOccupyingBlockAt(p: Pos): Boolean = ??? // check if p is in the body

  def isHeadCollision(other: Snake): Boolean = ??? // check if the heads collide

  def isTailCollision(other: Snake): Boolean = ??? // my head in someone else's tail

  private var _isEatenByMonster: Boolean = false
  def isEatenByMonster: Boolean = _isEatenByMonster
  def eatenByMonster(): Unit = ???

  def move(): Unit = ???
    // grow and shrink according to rules
    // actions if eats fruit or get eaten by monster

  override def toString = // bra vid println-debugging
    body.map(p => (p.x, p.y)).mkString(">:)", "~", s" going $dir")

  def draw(): Unit = ???

  def erase(): Unit = ???
```

Here is a sketch of the abstract class `SnakeGame` with the abstract methods `isGameOver` and `play` which are overridden in the subsequent subclasses `OnePlayerGame` and `TwoPlayerGame`:

```
package snake

object SnakeGame:
  enum State:
    case Starting, Playing, GameOver, Quitting
  export State.*

abstract class SnakeGame(settings: Settings) extends introprog.BlockGame(
  title                 = settings.windowTitle,
  dim                   = settings.windowSize,
  blockSize             = settings.blockSize,
  background            = settings.background,
  framesPerSecond       = settings.framesPerSecond,
  messageAreaHeight     = settings.messageAreaHeight,
  messageAreaBackground = settings.messageAreaBackground
):
  // examples of different visibility (discuss choice of visibility based on use)
  var entities: Vector[Entity] = Vector.empty
  protected var players: Vector[Player] = Vector.empty
  private var isPaused = false

  import SnakeGame.*

  protected var state: State = Starting
```

```
private var _iterationsSinceStart = 0
def iterationsSinceStart = _iterationsSinceStart

def enterStartingState(): Unit = ??? //erase, announce "press space to start"

def enterPlayingState(): Unit = ??? //erase, for each entity: reset & draw

def enterGameOverState(): Unit = ??? // inform "game over"

def enterQuittingState(): Unit =
  println("Goodbye!")
  pixelWindow.hide()
  state = Quitting

def randomFreePos(): Pos =
  ??? // pull random-pos until free space, used by fruit, monster

override def onKeyDown(key: String): Unit =
  println(s"""key "$key" pressed""")
  state match
    case Starting => if key == " " then enterPlayingState()

    case Playing =>
      if key == "Esc" then
        println(s"Toggle pause: isPaused == $isPaused")
        isPaused = !isPaused
      else
        players.foreach(_.handleKey(key))

    case GameOver =>
      if key == " " then enterPlayingState()
      else if key == "Esc" then enterQuittingState()

    case _ =>

override def onClose(): Unit =
  println("Window Closed!")
  enterQuittingState()

/** Implement this with logic for when to end the game */
def isGameOver: Boolean

/** Override this if you want to add game-logic in gameLoopAction
 *  Call super.onIteration() if you want to keep the step counter.
 */
def onIteration(): Unit =
  clearMessageArea()
  drawTextInMessageArea(s"Number of steps: $iterationsSinceStart", 10, 2)

override def gameLoopAction(): Unit =
  if state == Playing && !isPaused then
    _iterationsSinceStart += 1
    entities.foreach(_.erase())
    entities.foreach(_.update())
    entities.foreach(_.draw())
    onIteration()
    if isGameOver then enterGameOverState()

final def start(ps: Player*)(es: Entity*): Unit =
```

```
    players = ps.toVector
    entities = es.toVector
    isPaused = false
    pixelWindow.show()  // enables restart even if the window has been closed
    enterStartingState()
    gameLoop(stopWhen = state == Quitting)

  /** Implement this with a call to start with specific players and entities. */
  def play(playerNames: String*): Unit
```

If the group considers using git and GitHub:

- Discuss in the group whether everyone has sufficient knowledge to work with git and GitHub, as well as pros and cons of doing so.
- If not all are comfortable with git and GitHub, consider whether you want to do manual version control with copying new files via USB drive, ssh or uploading/downloading via cloud storage. After experiencing manual version control concretely, you gain a deeper understanding of the need for tool support for version control, and it is more motivating to learn git.
- Discuss workflows. How will you use GitHub issues, git branches, etc.? Or should everyone push to the main branch? Will you use GitHub pull requests, reviews, etc.?
- Check that you have a `.gitignore` before pushing, so that e.g. binary files do not end up in your repo, which can cause tricky cleanup work and unnecessary merge conflicts. An example of a suitable `.gitignore` is here: https://github.com/lunduniversity/introprog/blob/master/workspace/w10_snake/.gitignore
- **Be careful to make your GitHub repo private!** It is not permitted to share lab solutions on the internet – doing so may result in disciplinary action for assisting cheating, and you may be suspended from your studies.

Chapter 11

Lab: `snake1`

Goals

- ☐ See goals in last week's exercise. This lab extends over two weeks. Last week's presentation concerned the work plan and ongoing development, as well as your individual responsibility. This week, the completed solutions shall be presented by the respective main responsible individual.

Preparations

- ☐ Do exercise `Excercise11` in section 11.

11.1 Group Lab Presentation

1. Explain the overall structure of the code.
2. Describe which parts you have contributed to in the code.
3. Give a brief explanation of concepts you have practiced.
4. Account for what you have learned about the challenges of system development in a group.
5. Account for the feedback you have received from reviews and how you have worked to improve readability during your stepwise extensions of your code.
6. Account for how and what feedback you have given when you have reviewed someone else's code.

Chapter 12

Project

12.1 Project Assignment: bank

Focus

☐ Be able to implement a complete program based on a given specification
☐ Be able to combine different parts from different modules
☐ Understand how Java classes can be used in Scala
☐ Understand and assess when immutable/mutable as well as var/val should be used in larger contexts
☐ Be able to use companion objects
☐ Be able to read from and write to files
☐ Be able to search in different data structures in various ways

Background

In this project, you are to write a program that keeps track of bank accounts and customers in a bank. Besides tracking the bank's current state, the program should also keep a history of all state changes. The history should be detailed enough that the current state can be reconstructed by replaying all changes stored in the history. The program should be entirely text-based, i.e., interaction with the program is done via the terminal where a menu is displayed and input is received via the keyboard. You should write most of the program yourself, without any pre-written code. The program must follow the specifications given in the task, as well as the object-oriented principles you have learned in the course.

Requirements

The requirements for the bank application are listed below. To pass this assignment, all requirements must be met:

- The program must have the following menu options:
  - 1. Find accounts for a certain account holder with a given ID.
  - 2. Search for customers by (part of) name.
  - 3. Deposit money into an account.
  - 4. Withdraw money from an account.
  - 5. Transfer money between two different accounts.
  - 6. Create a new account.
  - 7. Remove an existing account.
  - 8. Print all bank accounts, sorted alphabetically by account holder.
  - 9. Print the history of all changes to the bank's state.
  - 10. Restore the bank to the state it had on a given date. For simplicity, you must permanently discard all history created after the date to which the bank is restored.
  - 11. Exit.
- When any of the following occur, the program must note this in the history:
  - Money is deposited into an account.

  - Money is withdrawn from an account.
  - Money is transferred between two accounts.
  - An account is created.
  - An account is removed.

- The history must be saved both in memory and in a file.
- When the program starts, it must read the history file to reconstruct the bank's previous state.
- The format of the history file must be the same as in this example file: https://github.com/lunduniversity/introprog/blob/master/workspace/w13_bank_proj/bank_log.txt
- Everything related to the user interface (such as prints to the terminal and input reading from the terminal) must take place in `BankApplication` or helper classes to `BankApplication`, not in any other classes specified in the assignment.
- All methods and attributes must have appropriate visibility, so that internal, mutable parts are not unnecessarily exposed.
- The choice of val/var and immutable/mutable must be appropriate.
- The program must work as in the attached examples of program execution.
- Reasonable error handling must be present. That is, it is desirable that the program does not crash when the user inputs invalid data, but instead informs the user that the input is invalid. However, you can assume that the history file is always in the correct format.
- The program design must follow the specifications given below.
- It is sufficient that the bank can handle integers, but this must be done with the `BigInt` class to allow large amounts. If your bank handles decimals, this must be done with `BigDecimal` to avoid rounding errors.
- The `BankAccount` class must generate a unique account number for each account. These must be restored if the bank's state is restored to an earlier date, i.e., if a restoration removes an account, its account number should become available again.
- The only way to change the state of a `Bank` must be (besides calling `returnToState`) by calling `doEvent` with a `BankEvent` that describes the state change. At first glance, this may seem a bit cumbersome, but when implementing the change history, it will be very helpful that there exists a `BankEvent` representing each change.
- For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

Design

Below follows a description of members that the different classes in the bank application must contain. These starter classes exist in the course workspace, together with the pre-written classes `HistoryEntry` and `Date` as well as `BankEvent` with associated subtypes: https://github.com/lunduniversity/introprog/tree/master/workspace/w13_bank_proj

```
package bank

/**
 * A customer of a bank with provided name and id.
 */
case class Customer(name: String, id: Long):
  override def toString(): String = ???
```

```
package bank

/**
 * A bank account for hold by a specific customer.
 * The account is given a unique account number and initially
 * has a balance of 0 kr.
 */
class BankAccount(val holder: Customer):
  val accountNumber: Int = ???

  /**
   * Deposits the provided amount in this account.
   */
  def deposit(amount: BigInt): Unit = ???

  /**
   * Returns the balance of this account.
   */
  def balance: BigInt = ???

  /**
   * Withdraws the provided amount from this account,
   * if there is enough money in the account. Returns true
   * if the transaction was successful, otherwise false.
   */
  def withdraw(amount: BigInt): Boolean = ???

  override def toString(): String = ???
```

```
package bank

import time.Date

/**
 * A bank with no accounts and no history.
 */
```

```
class Bank():
 /**
   * Returns a list of every bank account in the bank.
   * The returned list is sorted in alphabetical order based
   * on customer name.
   */
  def allAccounts(): Vector[BankAccount] = ???

  /**
   * Returns the account holding the provided account number.
   */
  def findByNumber(accountNbr: Int): Option[BankAccount] = ???

  /**
   * Returns a list of every account belonging to
   * the customer with the provided id.
   */
  def findAccountsForHolder(id: Long): Vector[BankAccount] = ???

  /**
   * Returns a list of all customers whose names match
   * the provided name pattern.
   */
  def findByName(namePattern: String): Vector[Customer] = ???

  /**
   * Executes an event in the bank.
   * Returns a string describing whether the
   * event was successful or failed.
   */
  def doEvent(event: BankEvent): String = ???

  /**
   * Returns a log of all changes to the bank's state.
   */
  def history(): Vector[HistoryEntry] = ???

  /**
   * Resets the bank to the state it had at the provided date.
   * Returns a string describing whether the event was
   * successful or failed.
   */
  def returnToState(returnDate: Date): String = ???
```

Tips

- Use a `match` expression to handle the different subtypes of `BankEvent` when implementing `doEvent`.

```
event match {
  case Deposit(account, amount) => ???
  case Withdraw(account, amount) => ???
```

```
  case Transfer(accFrom, accTo, amount) => ???
  case NewAccount(id, name) => ???
  case DeleteAccount(account) => ???
}
```

- To write to a file easily, you can for example use static methods in the `Files` class available in `java.nio.file`. To avoid portability issues, you can use a fixed `Charset`, e.g., `UTF_8`, which is available in `java.nio.charset.StandardCharsets.UTF_8`.
- To read from a file you can use `introprog.IO`. Pay particular attention to the method `appendString` and how newline characters are handled in `appendLines` https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala
- Be thorough in that your tests cover more cases than the provided examples (see 8), as these may be needed for more extensive testing and debugging and may be requested during demonstrations.

Mandatory tasks

Task 1. Implement the class `Customer`. Test that it works in the REPL.

Task 2. Implement the class `BankAccount`. Test that it works in the REPL.

Task 3. Create a main program in the singleton object `BankApplication`. Make sure the main program prints the menu correctly and can read input from the keyboard corresponding to the menu choices. Let menu selections give a message informing the user that they are not yet implemented.

Task 4. Implement the class `Bank`.

a) Implement menu choice 6. When the user chooses to create a new account, `BankApplication` should create a `NewAccount` object which it then uses as an argument in a call to `doEvent` in `Bank`. It is in `doEvent` (or a private function called from `doEvent`) where the account is actually created.

b) Implement menu choice 8. Verify that both menu choices 6 and 8 work correctly.

c) Implement menu choice 9. Every time `doEvent` executes without error, its `BankEvent` argument should be added to the history along with the current date.

d) Implement all other menu options, except menu option 10. Test the new menu options carefully as you implement them, especially so that the change history works correctly. Make extensions you consider necessary.

Task 5. Implement backup of the history.

a) When something is added to the history, it must also be immediately written to a history file. The bank must not have to be closed for the output to be recorded in the file; if this were the case, information could be lost if the bank crashes. Use the `toLogString` method in `HistoryEntry` to get output in the correct format.

b) When the program starts, it must read all events from the history file and replay them one by one. This way the bank's state can be restored, even though only the change history is saved and not the state itself. Use the `fromLogString` method in `HistoryEntry` when reading strings from the file.

Task 6. Implement menu choice 10 by first resetting the bank's state and then replaying everything in the history that happened before the given date. The rest of the history should be permanently removed, both in memory and in the history file.

Optional extra tasks

First complete the mandatory parts of the project. Then, if you wish, you can extend your program as follows.

Task 7. Implement a new menu option that prints all account history for a given person. The history should include the type of event with associated parameters, the balance at the time of the event, as well as the date of the event. (This can be useful when testing your program.)

Task 8. Write one or more of the classes `Customer` and `BankAccount` in Java instead and use these in your Scala code.

Example of history file

An example history file is included in the workspace directory for this project assignment. Reading and printing must be done using its format. Each line represents an event, and the format for a line is: **Year Month Day Hour Minute BankEventType Argument**. The different kinds of `BankEvent` are represented by the following letters: D for `Deposit`, W for `Withdraw`, T for `Transfer`, N for `NewAccount`, and E for `DeleteAccount`.

Example of program execution

Below are possible examples of running the program. Data entered by the user is marked in bold. Your program does not have to look exactly the same, but the overall structure as well as the execution result must be the same. When the first example starts, the bank is assumed to have no accounts.
The list of options, which is italicized in the first example, is not printed in later examples to save paper. Your program should always print the list of options before the user is to enter a choice.

- -
1. Find accounts for a given customer
2. Search for customers by (part of) name
3. Deposit money
4. Withdraw money
5. Transfer money between accounts
6. Create new account
7. Delete existing account
8. Print all accounts in the bank
9. Print change history
10. Restore the bank to an earlier date
11. Exit
Choice: **6**
Name: **Adam Asson**
Id: **6707071234**
New account created with account number: 1000
10:03 14/5-2016

- -
Choice: **1**
Id: **6707071234**
Account 1000 (Adam Asson, id 6707071234) 0 kr
10:04 14/5-2016

- -
Choice: **6**
Name: **Berit Besson**
Id: **8505255678**
New account created with account number: 1001
10:12 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 0 kr
Account 1001 (Berit Besson, id 8505255678) 0 kr
10:13 14/5-2016

- -
Choice: **2**
Name: **adam**
Adam Asson, id 6707071234
10:15 14/5-2016

- -
Choice: **6**
Name: **Berit Besson**
Id: **8505255678**
New account created with account number: 1002
13:56 14/5-2016

- -
Choice: **2**
Name: **erit**
Berit Besson, id 8505255678
14:01 14/5-2016

- -
Choice: **3**
Account number: **1000**
Amount: **5000**
Transaction successful.
14:36 14/5-2016

- -
Choice: **5**
Account number to transfer from: **1000**
Account number to transfer to: **1001**
Amount: **1000**
Transaction successful.
14:37 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 4000 kr
Account 1001 (Berit Besson, id 8505255678) 1000 kr
Account 1002 (Berit Besson, id 8505255678) 0 kr
14:52 14/5-2016

- -
Choice: **7**
Enter account to delete: **1002**
Transaction successful.
14:54 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 4000 kr
Account 1001 (Berit Besson, id 8505255678) 1000 kr
14:55 14/5-2016

- -
Choice: **9**
10:03 14/5-2016: Created an account belonging to Adam Asson, id 6707071234
10:12 14/5-2016: Created an account belonging to Berit Besson, id 8505255678
13:56 14/5-2016: Created an account belonging to Berit Besson, id 8505255678
14:36 14/5-2016: Deposited 5000 kr in account 1000
14:37 14/5-2016: Transferred 1000 kr from account 1000 to account 1001
14:54 14/5-2016: Deleted account 1002
14:58 14/5-2016

- -
Choice: **10**
Which date do you want to restore the bank to?
Year: **2016**
Month: **5**
Day: **14**
Hour: **10**
Minute: **5**
Bank restored.
15:00 14/5-2016

- -
Choice: **9**
10:03 14/5-2016: Created an account belonging to Adam Asson, id 6707071234
15:00 14/5-2016

- -

Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 0 kr
15:01 14/5-2016

- -

Choice: **3**
Account number: **1001**
Amount: **5000**
Transaction failed. No such account found.
15:06 14/5-2016

- -

Choice: **4**
Account number: **1000**
Amount: **1000**
Transaction failed. Insufficient balance.
15:23 14/5-2016

12.2 Project Assignment: `music`

Preparations

- ☐ Test that the computer you will use for the presentation can play sound with `javax.sound.midi` by running `Main` in the provided code.[1]
- ☐ It is good if you can bring headphones to the computer lab so that you do not disturb others.
- ☐ Download the given code via the course's GitHub location or via the course website under Links.

Background

When writing programs, one often creates models of a certain real *domain*, which can be e.g. the rules of the social insurance office or a physics simulation in a computer game. To be able to create such models, one often needs to acquire *domain knowledge* by carefully understanding what different concepts in the domain mean and how they are related. With this knowledge, you can create code that models the domain, based on carefully chosen simplifications of the complex reality. The ability to create domain models forms an important foundation for the art of developing good software systems, and you will learn more about this in upcoming courses.
In this lab assignment, you will create a program based on a simplified model of the domain *music*. You will receive ready-made code that models how notes are built, and how different string instruments work. With this domain model, you will create your own music program that uses *chords* which consist of several notes played together.

Domain model

Pitch

A **note** (eng. *note*) played on an instrument, e.g. a piano or a guitar, has a **pitch** (eng. *pitch*) that is related to the specific fundamental frequency of the note's sound. In our model of the music domain, we assign different distinct pitches a unique integer. A pitch can then be described by a `case class Pitch(nbr: Int)` where we use `nbr` in the range `(0 to 127)`. The integer `60` corresponds to a certain note, which also has the name `"C5"`, and which lies approximately in the middle of the keyboard on a piano. In (Western) music, one starts from 12 different *pitch classes* (eng. *pitch classes*). These twelve pitch classes are ordered in a sequence of so-called *half steps* and have the following **pitch class names**:

```
val pitchClassNames: Vector[String] =
  Vector("C","C#","D","D#","E","F","F#","G","G#","A","A#","B")
```

After the pitch class named `B`, the pitch class named `C` returns. The symbol `#` represents a raising by a half step. The pitch class `C#` is pronounced *ciss* in Swedish, and *see sharp* in English.[2] Note that there is a half step between `E` and `F`, and between `B` and `C` (therefore there are neither `E#` nor `B#` in the list of pitch class names.[3])

[1] At the time of writing, sound does not work under WSL (Windows Subsystem for Linux). Feel free to search online for possible solutions to sound issues in WSL.

[2] The flat sign ♭ is also used, which is pronounced *flat* in English, for lowering a note by a half step, but for simplicity we disregard this naming convention in our model.

[3] You can read more about why this is the case on e.g. Wikipedia, but you may also just accept that it is simply like this due to historical reasons.

On a piano, the white keys correspond to the pitch class names `C D E F G A B` and the black keys correspond to the pitch class names `C# D# F# G# A#`.
A so-called **pitch class** is a positive integer in the interval `0 until 12` corresponding to the index of the pitch class name in `pitchClassNames`. A pitch `Pitch(nbr)` belongs to the pitch class `nbr \% 12`.
Using integer division by 12, one can find the pitch's so-called **octave**, i.e. `nbr / 12`. A piano normally spans 7 or 8 octaves. A pitch `Pitch(nbr)` can also be named by combining the pitch class name and the pitch's octave, e.g. `"C5"`.
With this domain description, we can create a more detailed model of the concept of pitch using a case class and accompanying companion object:

```
package music

case class Pitch(nbr: Int):  //Tonhöjd
  assert((0 to 127) contains nbr, s"Error: nbr $nbr outside (0 to 127)")
  def pitchClass: Int        = nbr % 12
  def pitchClassName: String = Pitch.pitchClassNames(pitchClass)
  def name: String           = s"$pitchClassName$octave"
  def octave: Int            = nbr / 12
  def +(offset: Int): Pitch  = Pitch(nbr + offset)
  override def toString      = s"Pitch($name)"

object Pitch:
  val defaultOctave = 5  // mittenoktaven på ett pianos tangentbord

  val pitchClassNames: Vector[String] =
    Vector("C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B")

  val pitchClassIndex: Map[String, Int] = pitchClassNames.zipWithIndex.toMap

  def fromString(s: String): Option[Pitch] = scala.util.Try {
    val (pitchClassName, octaveName) = s.partition(c => !c.isDigit)
    val octave = if octaveName.nonEmpty then octaveName.toInt else 5
    Pitch(pitchClassIndex(pitchClassName) + octave  * 12)
  }.toOption

  def apply(s: String): Pitch =
    fromString(s).getOrElse(throw new IllegalArgumentException)
```

The companion object has two factory methods that can create `Pitch` objects from a string representation of a pitch.

- The method `fromString` converts a string to an `Option[Pitch]`.
- The method `apply` throws an exception if it is not possible to convert a string to a `Pitch` object.

✎ **Task 1.** Which two expressions in the `Try` block can cause the exceptions `NumberFormatException` and `NoSuchElementException` respectively? Investigate similar expressions in REPL that cause these exceptions. How can the factory method `fromString` be rewritten to use `toIntOption` instead of `toInt` on the string and `get` instead of `apply` on the key-value table and without using `scala.util.Try`? What are the disadvantages of detouring via `scala.util.Try` instead of methods that directly produce `Option`?

Task 2. Investigate the class `Pitch` in REPL.

```
> sbt
sbt> console
Welcome to Scala 2.13.3 (OpenJDK 64-Bit Server VM, Java 11.0.8).
Type in expressions for evaluation. Or try :help.

scala> import music._
import music._

scala> Pitch("C#5").nbr
res0: Int = 61

scala> Pitch("C") + 1
res1: music.Pitch = Pitch("C#5")
```

a) Give an example of an argument to `Pitch.apply` that causes an exception to be thrown. ✎

b) Give an example of an argument to `Pitch.fromString` that returns `None`. ✎

c) Give an example of an argument to `Pitch.+` that causes an exception to be thrown. ✎

Task 3. Modify the implementation of `fromString` so that instead of `.toOption`, you perform pattern matching with `match` on the `Try` results `Success` and `Failure` in separate case clauses of the form `case Success(x) =>` and `case Failure(e: ???) =>` `???`, returning the appropriate value. Only handle the two expected exception types you identified in task 1. Ensure that all other possible exceptions are thrown through this clause: `case Failure(e) => throw e`

a) Test that your solution works both in normal cases and when given an incorrect pitch name.

b) Investigate what happens if you comment out various case clauses. When does the compiler warn? Why? ✎

c) Are there any advantages or disadvantages to catching only certain exceptions? ✎

Chords

A chord consists of several notes played together. One can play a chord on a string instrument by striking a set of notes simultaneously or a sequence of notes in quick succession. One chooses to call one of the notes in the chord (usually the lowest/first note) the **root** (eng. *root*).
An **interval** is a note's relative pitch distance from the root. Chords have different names depending on which intervals are included in the chord. There are many different chord names, but here we limit ourselves for simplicity to four different chord types: [4]

- major chord, denoted e.g. `"C"`,
- minor chord, denoted e.g. `"Cm"`,
- seventh chord, denoted e.g. `"C7"`,
- major seventh chord, denoted e.g. `"Cmaj7"`.

[4] If you want to learn more about chord names, read here: https://en.wikipedia.org/wiki/Chord_(music)

In the case class `Chord` below there is a method `name` that defines which intervals are included in the different chord types above, except major seventh chords. The complicated modulo-12 calculation before the matching ensures that intervals in different octaves are treated equally, even for negative intervals.

```
package music

case class Chord(ps: Vector[Pitch]):
  assert(ps.nonEmpty, "Chord pitch sequence is empty")

  val pitchClasses: Vector[Int]   = ps.map(_.pitchClass).toVector

  def apply(i: Int): Pitch = ps(i)

  def intervals(root: Pitch = ps(0)): Vector[Int] = ps.map(_.nbr - root.nbr)

  def relativePitchClasses(root: Pitch = ps(0)): Vector[Int] =
    intervals(root).map(i => (i%12 + 12) % 12).distinct.sorted

  def name(root: Pitch = ps(0)): String = relativePitchClasses(root) match
    case Vector(0, 4, 7)     => root.pitchClassName
    case Vector(0, 3, 7)     => root.pitchClassName + "m"
    case Vector(0, 4, 7, 10) => root.pitchClassName + "7"
    case _ => root.pitchClassName + intervals(root).mkString("[",",","]")

  override def toString = ps.map(_.name).mkString("Chord(",",",")")

object Chord:
  def apply(xs: String*): Chord = Chord(xs.map(Pitch.apply).toVector)

  def random(pitchNumbers: Seq[Int] = (60 to 72), n: Int = 3): Chord =
    val shuffled = scala.util.Random.shuffle(pitchNumbers).toVector
    Chord(shuffled.take(n).map(Pitch.apply))
```

Task 4.

a) Major seventh chords have the same intervals as seventh chords, except that the fourth interval should be `11` half steps from the root instead of `10`. Add a case clause in `Chord.name` so that major seventh chords get a name ending with the suffix `"maj7"`.

b) Test your code and check that the chord `Chord("D4","F#4","A4","C#5")` gets the name `"Dmaj7"`

```
scala> Chord("D4","F#4","A4","C#5").name()
res2: String = "Dmaj7"
```

c) Which four notes does a `Cmaj7` chord with root `"C5"` consist of?

String instruments

A string instrument, e.g. a piano or a guitar, is characterized by the ability to play chords because several strings can be vibrated so that many notes are played together. In our model, we capture this feature with a trait `StringInstrument` that has a method `toChordOpt` which returns some chord if at least one string is played.

Guitar and ukulele are examples of string instruments with a fretboard. One plays a fretted instrument by pressing strings against the fretboard with one hand while plucking the strings with the other. Different instances of these instruments may differ in the number of strings and how these strings are tuned. A normal guitar has 6 strings, while a normal ukulele only has 4 strings. These features are modeled in the code below.

Each string has a tuning peg by which one can change the string's tension, the string's so-called **tuning** (eng. *tuning*). If you pluck all open strings on a guitar with standard tuning, the tones E3, A3, D4, G4, B4, E5 are played, counted from the thickest to the thinnest string.

```
package music

trait StringInstrument { def toChordOpt: Option[Chord] }

case class Piano(isKeyDown: Set[Int]) extends StringInstrument:
  def toChordOpt: Option[Chord] =
    if isKeyDown.nonEmpty then
      Some(Chord(isKeyDown.toVector.sorted.map(Pitch.apply)))
    else None

trait FrettedInstrument extends StringInstrument:
  def nbrOfStrings: Int
  def tuning: Vector[Pitch]
  def grip:   Vector[Int]
  def toChordOpt: Option[Chord] =
    val notes =
      for i <- grip.indices if grip(i) >= 0
      yield tuning(i) + grip(i)
    if notes.nonEmpty then Some(Chord(notes.toVector)) else None

case class Guitar(pos: (Int,Int,Int,Int,Int,Int)) extends FrettedInstrument:
  val grip = Vector(pos._1, pos._2, pos._3, pos._4, pos._5, pos._6)
  val nbrOfStrings = 6
  val tuning =
    "E3 A3 D4 G4 B4 E5".split(' ').map(Pitch.apply).toVector

case class Ukulele(pos: (Int,Int,Int,Int)) extends FrettedInstrument:
  val grip = Vector(pos._1, pos._2, pos._3, pos._4)
  val nbrOfStrings = 4
  val tuning =
    "A5 D5 F#5 B5".split(' ').map(Pitch.apply).toVector
```

If you press the fretboard at different positions, you get different notes, depending on which position you press. The positions on the fretboard are counted from one and upwards. Position `0` corresponds to a **open string**, meaning that you pluck a string without pressing down on the fretboard over that string. A negative position, e.g. `-1`, indicates that a string is not played at all; many guitar chords are played by only plucking a subset of the strings. An example of a guitar chord is shown in figure 12.1.

Task 5. Study the model of string instruments above and use REPL to answer these questions: ✎

Figure 12.1: A C major chord on a guitar corresponding to `Guitar(3,3,2,0,1,0)`.

a) What is the name of this piano chord if we choose the lowest note in the chord as the root: `Piano(Set(60, 64, 67, 70))`

b) What are the names of the notes included in the chord `Guitar(3,3,2,0,1,0)`?

c) What is the name of this chord with root A: `Ukulele(0,2,1,2)`

Electronic instruments

An electronic instrument synthesizes sound using analog and/or digital electronics, and is therefore called a **synthesizer**, often abbreviated *synth* (eng. *synth*).
Most modern PC operating systems include software-implemented synths that follow the so-called MIDI standard. The Java package `javax.sound.midi` contains classes that can make such a MIDI synth play music.
The MIDI standard is based on a model of a piano keyboard where different notes can be "on" or "off" depending on whether a key is pressed or not. These notes' pitches are modeled similarly to our class `Pitch`, where pitch `60` corresponds to the note `"C5"`, etc. A key can be pressed with varying force, represented by an integer value in `Range(0,128)` called `velocity`. A high value produces a strong note, while a low value corresponds to a weak (quiet) note.
A synth following the MIDI standard can play sound via 16 different so-called **channels** (eng. *channel*), numbered `(0 until 16)`, where each channel can be set to play a sound resembling a certain real instrument, such as piano or guitar.
In the course workspace in the package `music` there is a `Synth` module that simplifies the use of the Java package `javax.sound.midi`. The module `Synth` has the method `playBlocking` which can play multiple notes for a certain duration using the synth on your sound card. The execution of your program is blocked until the notes have finished playing, hence the *"blocking"* in the name.
The method `playBlocking` has the following parameters, default arguments, and return type: [5]

[5] If you are curious, you can study the implementation of the `Synth` module here: https://github.com/lunduniversity/introprog/tree/master/workspace/w13_music_proj The

```
def playBlocking(
  noteNumbers: Seq[Int] = Seq(60), // a sequence of pitches
  velocity: Int         = 60,      // how hard the strike in Range(0, 128)
  duration: Long        = 300,     // how long in milliseconds
  spread:   Long        = 50,      // milliseconds between notes
  after:    Long        = 0,       // milliseconds before the first note
  channel:  Int         = 0        // MIDI channel that plays the notes
): Unit
```

Task 6. Call `playBlocking()` in REPL and check if your computer can play the note `"C5"`. Use headphones so you do not disturb your lab mates. Try what happens when giving different arguments to `playBlocking`.

Task 7. Complete the module `ChordPlayer` below so that the method `play` can play a chord. The case class `Strike` represents a chord strike.

```
package music

object ChordPlayer:

  case class Strike(
    velocity: Int         = 50,   // hur hårt anslag i Range(0, 128)
    duration: Long        = 500,  // hur länge i millisekunder
    spread:   Long        = 50,   // millisekunder mellan tonerna
    after:    Long        = 0     // millisekunder innan första tonen
  )

  def play(chord: Chord, strike: Strike = Strike(), channel: Int = 0): Unit =
    strike match
      case Strike(v, d, s, a) => ???
```

Task 8. Implement a singleton object named `Test` with a `main` method that, using your `play` method from the previous task, plays some different chords.

Task 9. Create a terminal app that can play chords. In the course workspace in `w13_music_proj` there is a started terminal app you can build on. It already has a `Main.main` method that starts a loop where the user can give commands (eng. *Command Line Interface, CLI*). The command `?` gives help and the command `:q` quits.

```
*** Welcome to music!
music> ?
?         print help
:q        quit this app
!         play chord TODO
music> !
play chord TODO
music> :q
Goodbye music!
```

code is easier to understand if you also read the API documentation of the package `javax.sound.midi` and learn more about the MIDI standard using e.g. Wikipedia.

As seen above, there is also a started command `!` which is intended to play a chord, but currently only prints a TODO message. Make it so that the user can play chords from different instruments using `!` as shown below:

```
music> ! p 60 64 67
Play Piano(Set(60, 64, 67)) Chord(C5,E5,G5)
music> ! g 0 2 2 0 0 0
Play Guitar((0,2,2,0,0,0)) Chord(E3,B3,E4,G4,B4,E5)
```

Task 10. Create a command that lets users define their own command names that can then be easily run using the defined name. If defined with an existing name, the old definition should be replaced.

```
music> def Em = ! g 0 2 2 0 0 0
defined Em = ! g 0 2 2 0 0 0
music> Em
Play Guitar((0,2,2,0,0,0)) Chord(E3,B3,E4,G4,B4,E5)
```

It should be possible to make nested def-commands, meaning the body of a def contains the name of another def. Try defining a whole song that consists of defined chords.

Task 11. For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

Optional tasks

Task 12. Make it so that definitions are saved between runs.

Task 13. Implement more optional commands. You can e.g.

- create a command that draws a fingering chart for guitar chords or finger positioning on piano with `introprog.PixelWindow`.
- create a command to make drumbeats, e.g. `drum mygrove x x o x` for 2 hi-hat + bass drum + 1 hi-hat and `start mygrove` and `stop mygrove` for playback of a beat in the background with `playConcurrently`. See `Synth.scala` for which instruments produce drum sounds based on their names `music.Synth.instruments.map(_.getName)`, e.g. `Standard Kit` or `Synth Drum`. Use channel 10 to play the drum sounds.

Task 14. Use the open-source project `jline` instead of `scala.io.StdIn.readLine` to automatically get up-arrow history, Ctrl+A Ctrl+K, TAB-completion, etc. See an example of usage of `jline` here: https://github.com/bjornregnell/termut

12.3 Project Assignment: `photo`

Background

This project involves implementing your own image processing application, a very simplified version of *Photoshop* or *Gimp*.
A digital image consists of a grid, a so-called matrix, of pixels, each with a certain color. When many small pixels are placed next to each other in a grid, they blend together visually, and the viewer perceives an image.
Images can be manipulated by applying various so-called *filters* that modify the image's pixels in interesting ways. You shall, based on the given mathematical theory, implement different filters using special matrix operations.
There are different systems for coloring pixels. For example, the CMYK system (cyan, magenta, yellow, black) is used for mixing color to be printed on paper or other materials. On a computer, however, the RGB system is usually used, which has the three primary colors red, green, and blue. The saturation of each primary color is specified by an integer which we henceforth assume lies in the interval [0, 255]. 0 denotes "no color" and 255 denotes "maximum color". Thus, one can represent 256 × 256 × 256 = 16,777,216 different shades of color. One can also represent grayscale images; this is done with colors that have the same value for all three primary colors: (0, 0, 0) is completely black, (255, 255, 255) is completely white.
In this project, we will create matrices of integers to compute interesting properties of an image, for example the intensity for each pixel. To save space when processing large images, we use integer matrices with the type `Short`, which use 16 bits in memory, instead of `Int`, which uses 32 bits in memory.

Preparations

In this project, the following parts of `introprog-scalalib` and `java.awt` are useful:

- `introprog.Image` for image handling.
- `introprog.PixelWindow` and `introprog.Dialog` for user interaction.
- `introprog.IO` for file handling.
- `java.awt.Color` for pixel color handling.

Read the documentation carefully for the classes in introprog here and do your own experiments in REPL so you understand how they can be used: https://fileadmin.cs.lth.se/pgk/api
Read about the class `java.awt.Color` here:
https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/java/awt/Color.html Download and study carefully the code given for this project here:
https://github.com/lunduniversity/introprog/tree/master/workspace/

Matrix with values of type `Short`

Task 1. `Matrix`. In the given code file `Matrix.scala` there are helper functions for creating and updating matrices with values of type `Short`, to save memory with large images.
Complete the missing implementations and test thoroughly in REPL so that everything works as it should before you proceed. *Tip:* You will find `Array.tabulate` useful.

```
scala> import photo.*

scala> val m = Matrix(3,3)(1,2,3,4,5,6,7,8,9) // a 3x3 matrix with Short values
val m: photo.Matrix = Array(Array(1, 4, 7), Array(2, 5, 8), Array(3, 6, 9))

scala> m(0,1)
val res0: Short = 4

scala> m(1,0) = 42

scala> m
val res1: photo.Matrix = Array(Array(1, 4, 7), Array(42, 5, 8), Array(3, 6, 9))

scala> m.row(0)
val res2: Array[Short] = Array(1, 42, 3)
```

User Interface

When the app starts, a window is displayed according to fig. 12.2, implemented by the given code in `Main.scala`. With the help of the given `Button.scala`, a column with clickable buttons is created. Study the code in `Main.scala` and `Button.scala` carefully so that you understand what happens. So far, no ImageEditor window opens, but that is part of the next task.

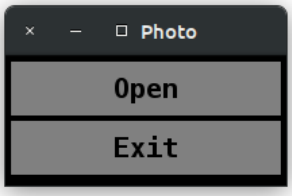


Figure 12.2: The Photo application's start window.

Task 2. `ImageEditor`. The following requirements shall be implemented:

- You shall create a code file `ImageEditor.scala` containing a class of the same name that implements an image editing window with a column of buttons on the left and an image loaded from a file on the right, as shown in fig. 12.3.
- When pressing the Exit button, a warning question "Ok to Exit without save?" shall be displayed using `introprog.Dialog.isOK` and the user shall be able to cancel the exit. If exit is chosen anyway, this shall be done with `System.exit(0)` so that all possibly active windows and associated threads are terminated immediately.
- When pressing the Open button, a file shall be selected using `introprog.Dialog.file`. If there is a subdirectory named `images` in the current directory, the file chooser shall start there; otherwise in the current directory.
- After OK in file opening, an image shall open in an image editing window as shown in fig. 12.3 with the buttons Save, Undo, Close, plus a button for each

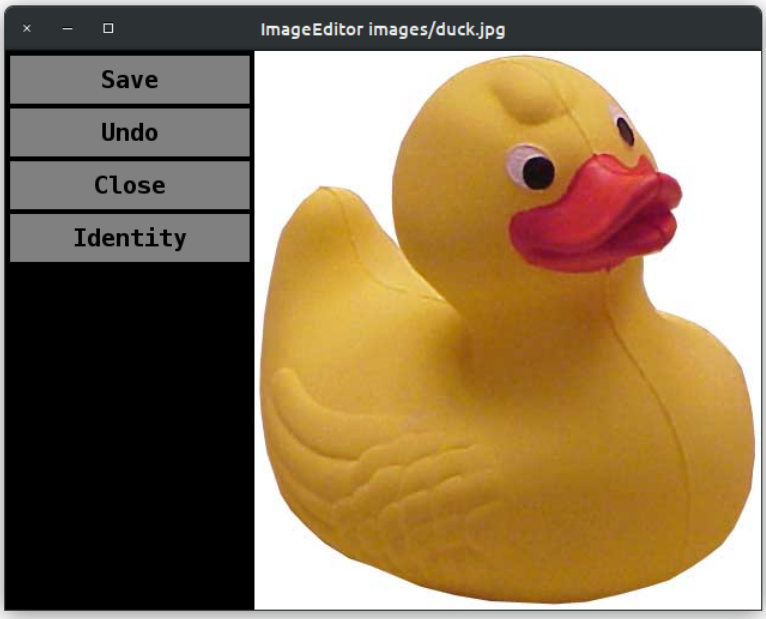


Figure 12.3: The image editing window before any filters (beyond the identity filter) are implemented. Each implemented filter should have a corresponding button.

filter, to the left of the image. The window's height and width shall be adjusted so that the entire image and all buttons fit.

- The main window and all image editing windows shall work in parallel. You shall achieve this by running the event loop in the `ImageEditor` class as an argument to the method `runInParallell` as below:

```
def runInParallell(block: => Unit) =
  new Thread{ override def run(): Unit = block }.start

def startEventLoop(): Unit = runInParallell:
  // initialization and event loop here
```

- It shall be possible to do Undo in multiple steps and restore all images before applying filters in turn. *Tip:* introduce an attribute `var history: Vector[Image]` initially containing the original image.

- The window title shall contain the name `ImageEditor` and the last two parts of the opened path, according to the example in fig. 12.3, since a full path such as `/home/userxyz/workspace/photo/images/duck.jpg` may not fit in the window's title bar.

- When saving, a filename query shall be asked with `introprog.Dialog.file` and checks made whether the file exists or not; if existing, a question with `introprog.Dialog.isOK` shall ask whether to overwrite or not.

- All implemented filters shall have a button that applies the filter and saves the result in history so that the filter can be undone with Undo. If the filter has arguments, an informative dialog shall open where the user can enter the

arguments via `introprog.Dialog.input`. The filter buttons shall be sorted alphabetically by filter name, see fig. 12.4.

Filters

You shall build upon the given code in `Filter.scala` shown below. You shall implement and test a number of different filters that modify images in interesting ways using different matrix algorithms.

```
package photo

import introprog.Image

trait Filter:
  def name: String

  def argDescriptions: Seq[String] = Seq()

  def nbrOfArgs = argDescriptions.length

  def apply(im: Image, args: Double*): Image

object Filter:
  val byIndex: Vector[Filter] = Vector(Identity)

  val byName: Map[String, Filter] = byIndex.map(f => f.name -> f).toMap

  def intensity(im: Image): Matrix = ???

  def convolve(p: Matrix, x: Int, y: Int, kernel: Matrix, weight: Int): Short = ???

  object Identity extends Filter:
    val name = "Identity"
    def apply(im: Image, args: Double*): Image =
      val result = Image.ofDim(im.width, im.height)
      result.updated((x, y) => im(x, y))
```

`Filter.scala` contains a base type for all filters with several members that all filters shall implement, according to the requirements below. There is one filter already implemented, `Identity`, which can be used for testing purposes; this filter simply copies all pixels to a new image and therefore has no editing effect.

- The method `apply` shall return a new image where the filter has been applied, without modifying the input image.
- A filter can have zero or more arguments of type `Double` that can influence what happens when the filter is applied. Each such argument shall have a short, instructive description in the sequence `argDescription`, in order.
- The method `intensity` shall calculate a so-called *intensity matrix* and is needed in implementing grayscale, Gauss, and Sobel filters. How an intensity matrix is calculated is described below.
- The method `convolve` shall perform a so-called convolution (averaging in matrices) and is needed in the implementation of the Gauss and Sobel filters. How convolution is performed is described below.

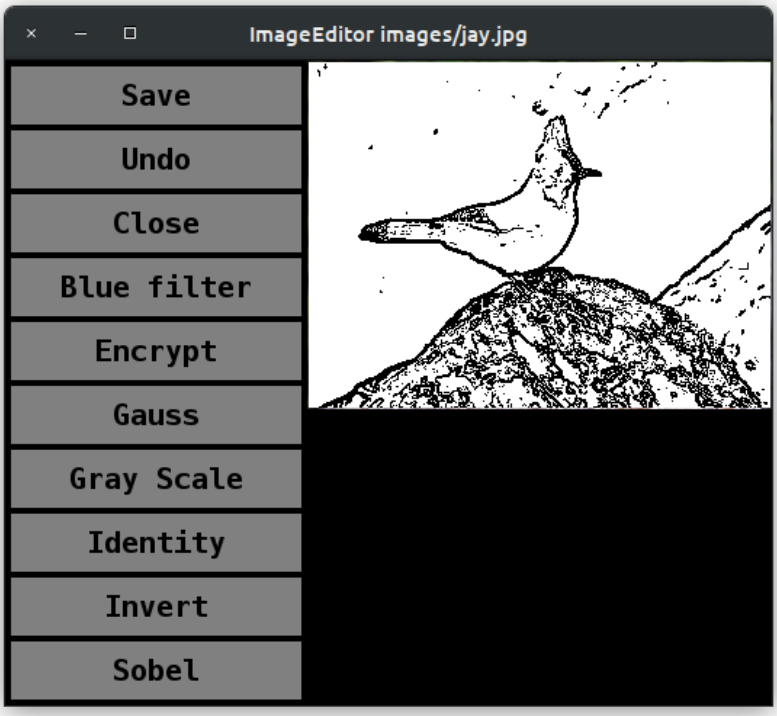


Figure 12.4: Image editing window with all filters implemented. A contour-enhancing so-called Sobel filter is applied with threshold value 150.

- All implemented filters shall be in the sequence `byIndex`, which is used in the table `byName`. These are needed to create all filter buttons and apply the respective filter.

You shall implement and test all filters in the tasks below, one at a time. The tasks are ordered in ascending difficulty.

Task 3. Blue filter. Create a filter in a singleton object named `Blue` which when applied gives a blue version of the image, where each pixel only contains the blue component. *Tip:* You have the method `getBlue` in the class `java.awt.Color`.

Task 4. Negative. Create a filter `Invert` that inverts an image, i.e. creates a "negative" copy of the image. Bright colors become dark and dark colors become bright. Consider what is meant by an inverted or negative copy. *Tip:* The new RGB values shall also be integers in the range 0 – 255. The new RGB values are *not* computed by any division expression over the old values (then the new values would be decimals and not integers in 0 – 255).

Task 5. Grayscale filter. Create a filter `GrayScale` that converts the image to grayscale. First implement the `intensity` method in `trait Filter` by averaging all three RGB components. Then use the intensity for each pixel to determine the grayscale level. If the intensity in a pixel, for example, is 105, then the new grayscale pixel should be a `Color` object with RGB values (105, 105, 105).

Task 6. Encryption. Create a filter `XOrCrypto` that encrypts the image with the xor operator `^`. This operator does a binary xor between the bits of an integer. For

example, 8 ^ 127 yields 119. If you do xor again with 127, i.e., 119 ^ 127, you get back 8. Each pixel is encrypted by applying xor between the original red, green, and blue values and random integers generated from a new instance of `scala.util.Random`. Three new random numbers are drawn for each pixel's RGB component from the same Random instance. Let the user give an argument that you use as the random seed when creating the `Random` instance. In this way, the image can be restored by applying the encryption filter again, with the same argument, on the now encrypted image.
If the filter's `argDescriptions` sequence is non-empty, `ImageEditor` shall prompt for each argument in turn and show each description in the dialog. The user's input is converted to a decimal number of type `Double` before the arguments are used in the `apply` method. Decide for yourself how to handle default values and error handling if the user enters a string that cannot be converted to `Double`. *Tip:* `toDoubleOption` and `getOrElse` will be helpful.

```
object XOrCrypto extends Filter:
  val name = "Encrypt"
  override val argDescriptions = Seq("Encryption key")

  def apply(im: Image, args: Double*): Image = ???
```

```
scala> Seq(8, 127, 8 ^ 127).map(_.toBinaryString)
val res0: Seq[String] = List(1000, 1111111, 1110111)

scala> 8 ^ 127
val res1: Int = 119

scala> 119 ^ 127
val res2: Int = 8
```

Task 7. Gauss filter. A Gauss filter makes the image a bit less sharp. Gauss filtering is an example of so-called *convolution filtering*. Convolution is a kind of localized averaging. New pixels are created by combining each pixel with its surrounding pixels according to a special matrix algorithm.
To achieve this, one uses a so-called *convolution kernel* K which is a small square integer matrix. Place K over each element in the intensity matrix and multiply each element of K with the corresponding element in the intensity matrix. Sum the products and divide the sum by the sum of the elements in K to obtain the new intensity value at that point (a kind of average). The division is done so that the new intensity falls within the correct range (0 – 255). Example:

$$intensity = \begin{pmatrix} 5 & 4 & 2 & 8 & \dots \\ 4 & 3 & 4 & 9 & \dots \\ 9 & 8 & 7 & 7 & \dots \\ 8 & 6 & 6 & 5 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \qquad K = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

The sum of the elements in K is $1+1+4+1+1=8$. To calculate the new intensity value at point `(1, 1)` where the current value is 3, calculate:

$$newintensity = \frac{0\cdot 5+1\cdot 4+0\cdot 2+1\cdot 4+4\cdot 3+1\cdot 4+0\cdot 9+1\cdot 8+0\cdot 7}{8} = \frac{32}{8} = 4$$

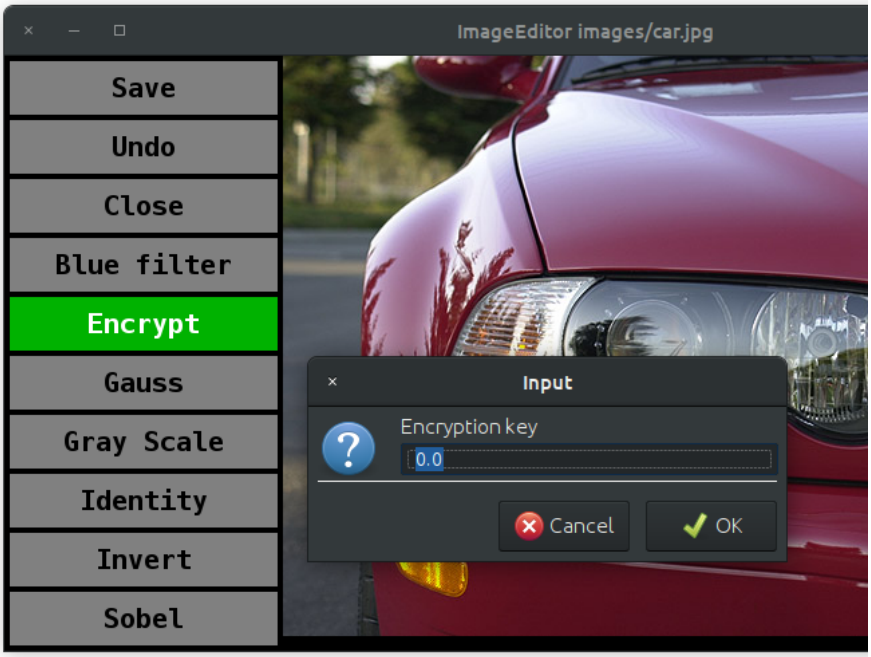


Figure 12.5: Encryption filter before application, during entering key.

Then continue by moving K one step to the right and calculate in the same way a new value for element with index `(1)(2)` (where the current value is 4 and the new value will be 5). Thereafter, do the same for all elements except for the "frame", i.e. the elements at the edges of the matrix.
Implement and test thoroughly first the method
`convolve(p: Matrix, x: Int, y: Int, kernel: Matrix, weight: Int): Short` in `trait Filter` which shall give the normalized sum of products of `kernel` and the points near `(x,y)` in matrix `p`, normalized by `weight`. `Tip: round` and `toShort` methods will be useful.

```
scala> import photo.*

scala> val p = Matrix(4,4)(5,4,2,8,4,3,4,9,9,8,7,7,8,6,6,5)
val p: photo.Matrix = Array(Array(5, 4, 9, 8), Array(4, 3, 8, 6), Array(2, 4, 7

scala> val K = Matrix(3,3)(0,1,0,1,4,1,0,1,0)
val K: photo.Matrix = Array(Array(0, 1, 0), Array(1, 4, 1), Array(0, 1, 0))

scala> Filter.convolve(p, 1, 1, K, K.flatten.sum)
val res0: Short = 4
```

Then create a filter `Gauss` which convolves using `convolve` for each color component separately. Proceed as follows:

1. Create three `short` matrices and store the red, green, and blue components of the pixels in them.

2. Perform convolution of the three components for each element and update `result` with the computed values.

3. The elements at the edges are not processed, but `result` must also have values for these. The simplest approach is to copy these elements unchanged from `im` to `result`. (Alternatively, you may set them to `Color.BLACK`, but then the filtered image might look smaller.)

Use the kernel `K` as above, and let `weight` be the sum of all elements in K.
It might be interesting to try values other than 4 in the center of the convolution matrix. With zero in the center, you get a stronger smoothing effect since the current pixel's value is ignored. Let the user enter an argument for the middle value, between 0 and 50, and describe this in `argDescriptions`. [6]

Task 8. Sobel filter. Sobel filtering is, like Gauss filtering, a type of convolution filtering. However, Sobel filtering gives the opposite effect compared to Gauss filtering, i.e. it enhances edges in an image. Essentially, it takes derivatives in the x and y directions and combines the result.

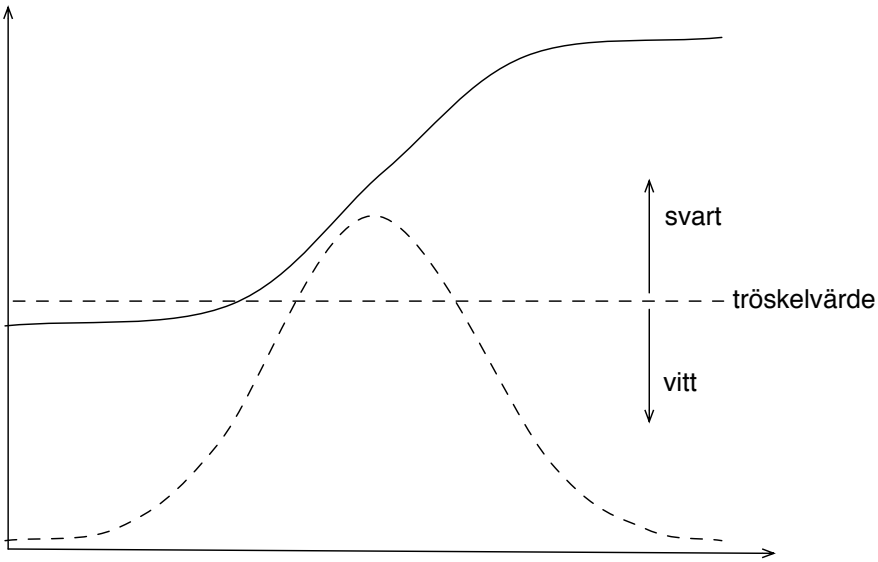


Figure 12.6: A function (solid line) and its derivative (dashed line).

In figure 12.6 a function f (solid line) and its derivative f' (dashed line) are shown. We see that where the function makes a "jump", the derivative has a large value. If the function represents the intensity of pixels along a line in the x or y direction, then those "jumps" correspond to edges in the image. If one then decides that pixels where the derivative value exceeds a certain threshold should be black and other pixels white, one obtains an image with strong edges.
Here the pixel intensity is not a continuous function that one can differentiate by ordinary mathematical rules. But it can be approximated by, for example, the formula:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

[6] It can sometimes be hard to see any difference between the Gauss-filtered image and the original. To get a really blurred image, you have to use a larger matrix as convolution kernel. You are encouraged to try this as an extra task.

As h approaches zero, this becomes the derivative definition. Further theoretical elaborations show that derivation can be expressed in a matrix with convolution as follows:

1. Calculate the intensity matrix with the method `intensity`.

2. For each point of the intensity matrix do two convolutions with these kernels:

$$SobelX = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \qquad SobelY = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

   Use the method `convolve` with weight 1. The coefficients in the matrix $SobelX$ express derivation in the x direction, while $SobelY$ expresses derivation in the y direction. Explaining why the coefficients are sometimes 1, sometimes 2, sometimes positive and sometimes negative requires studying the underlying theory closely, but we will not do that here.

3. If the convolution results at a point are denoted by `sx` and `sy` then an indicator for the presence of an edge is given by `math.abs(sx) + math.abs(sy)`. Absolute values are needed since there are negative coefficients in the convolution matrices.

4. Set the pixel to black if the indicator exceeds the threshold value, otherwise white. Let the threshold be an argument that the user can specify.

Create a filter `Sobel` that implements edge enhancement with the above algorithm. See example in fig. 12.4. The user shall specify the threshold value with argument description `"Threshold (0.0 - 255.0)"`.

Task 9. For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

Optional extra tasks

Task 10. Keyboard shortcut. Make it possible to apply filters via key presses. Extend the `trait Filter` so that all filters can have a shortcut. Write on the button what the shortcut is so that the user can discover it.

Task 11. Contrast filter. If one applies contrast filtering on a color image, the image will be converted to grayscale. (One can of course enhance contrast in a color image and get a color image as a result. Then one treats the three color channels separately.) Many images suffer from too low contrast. This is because the image does not use the full available range 0–255 for intensity. You get an image with better contrast if you "stretch" the interval as follows (linear interpolation):

```
val newIntensity = 255 * (intensity - 45) / (225 - 45)
```

As can be seen, a pixel with intensity 45 will get new intensity 0 and a pixel with intensity 225 will get new intensity 255. The intermediate pixels spread evenly over the interval `[0, 255]`. For pixels with intensity less than 45, set the new intensity to 0; for pixels with intensity greater than 225, set the new intensity to 255. We call the interval where most of the pixels are `[lowCut, highCut]`. Points with intensity less than `lowCut` are set to 0, points with intensity greater than `highCut` set to 255. For other points, interpolate with the formula above (replace 45 with `lowCut`, 225 with `highCut`).
Now remains the finding of suitable values for `lowCut` and `highCut`. This is not something that can be done simply, since the values depend on the intensity distribution of the pixels in the image. One therefore begins by computing the image's intensity histogram, i.e. how many pixels have intensity 0, how many have 1, ..., up to 255.
In most image processing programs, one can then look at the histogram and interactively decide values of `lowCut` and `highCut`. We won't do that here. Instead, we decide on a percentage `cutOff` that the user can enter as argument, which calculates `lowCut` such that `cutOff` percent of pixels have intensity less than `lowCut`, and `highCut` such that `cutOff` percent of pixels have intensity greater than `highCut`.

Example: Suppose an image contains 100,000 pixels and `cutOff` is 1.5. Compute the intensity histogram by registering each intensity in an integer array
```
val histogram = Array.ofDim[Int](256)
```
and compute `lowCut` so that
`histogram(0) + ... + histogram(lowCut) = 0.015 * 100000`
as close as possible, it probably won't be exactly equal. Compute `highCut` similarly.

Summary of the algorithm:

1. Compute the intensity matrix.
2. Compute the intensity histogram of the image.
3. User argument is used as `cutOff`.
4. Compute `lowCut` and `highCut` as above.
5. Compute the new intensity for each pixel according to the interpolation formula and store the new pixels in `result`.

Create a filter `Contrast` that implements the algorithm. In the *images* directory, the image *moon.jpg* might be suitable for testing, since it has low contrast. Note: if `cutOff` is 0, then this filtering produces the same result as `GrayScale`. This can be seen when studying the interpolation formula.

Task 12. Own filter. Create your own custom filter. For example, you can create a filter that takes five arguments, where the first two values represent an intensity interval and the last three values represent red, green, and blue components of a pixel which shall replace pixels whose intensity lies outside the given interval.

Task 13. Own interactive tools. Create optional interactive editing tools with mouse and keyboard input. Start with a selection tool that allows a rectangular portion of the image to be selected using the mouse. Make it possible to apply filters to the selected portion of the image. You can also make arguments for e.g. the Gaussian filter settable with a slider that you draw below the button and that can be adjusted with the mouse or arrow keys.